

记录想法

会话 ID: 019f2244-6a3e-7f51-853d-fa086f732b61 开始时间: 2026-07-02 11:59:20 消息数: 250

用户 2026-07-02 11:59:20

Files mentioned by the user:

Research_Proposal.pdf: C:/Users/XiaoyanXiong/Documents/rearch topic/Research_Proposal.pdf

My request for Codex:

这是我的idea

助手 2026-07-02 11:59:28

我先把这份 PDF 读一下，帮你抓住 idea 的核心和可能需要补强的地方。会用 PDF 技能来处理文档内容。

助手 2026-07-02 11:59:49

我现在先提取正文和页数，这样可以从研究问题、方法、数据集、创新点这些维度给你一个有用的反馈，而不是只看文件名猜。

助手 2026-07-02 12:00:10

提取到第 5 页时遇到 Windows 控制台编码问题，PDF 本身没坏。我会用 UTF-8 重新抽取一份临时文本，继续看完整内容。

助手 2026-07-02 12:05:43

完整正文已经抽出来了：18 页，约 4 万字符。我接下来会看结构大纲和方法部分，重点判断 idea 是否清晰、可做、创新点是否站得住。

助手 2026-07-02 12:06:32

内容上我已经能判断方向了。顺手再做一眼版面检查，看看那些被文本抽取成 “problemover / bothintra-flow” 的地方是真排版问题还是抽取器误读。

助手 2026-07-02 12:07:56

渲染工具的快捷命令找不到内部路径，先不让这个小工具耽误主线。我看正文内容已经足够给你研究方向反馈。

助手 2026-07-02 12:08:03

我会确认一下渲染依赖是否实际存在；如果没有，就只基于抽取正文给你反馈，不会装东西。

助手 2026-07-02 12:08:10

依赖目录在，但封装脚本可能有一个路径转发问题。这个不影响内容判断；我已经有 proposal 的完整结构和方法细节了。

助手 2026-07-02 12:08:27

我再试一次直接调用底层 exe，这样能避开封装脚本路径问题。只渲染第 2 页做抽样检查。

助手 2026-07-02 12:08:45

底层渲染成功了，我看一眼页面，确认有没有明显排版问题。

助手 2026-07-02 12:08:48

文件名带了补零编号，我查一下实际输出名。

助手 2026-07-02 12:09:01

版面抽样看起来正常；那些奇怪连词主要是文本抽取器的问题。不过第 14 页风险小标题可能真的缺少冒号/空格，我再抽样看一页。

助手 2026-07-02 12:09:15

渲染好了，我检查一下风险页的小标题排版。

助手 2026-07-02 12:09:42

我看完了。你的 idea 核心很清楚：做一个 **hierarchical time-aware Transformer for NIDS**，先在单个 flow 内建模 packet sequence、direction/timing/header metadata，再用第二层模型建模相邻 flows 的 context，目标是证明 packet-level temporal structure 和 cross-flow context 比传统 flow aggregation 更有信息量。

我觉得这个方向是成立的，而且 thesis framing 不错：重点不是“Transformer 更强”，而是 **network traffic 本身是 hierarchical + temporal 的，模型应该匹配这个结构**。这个论点比单纯刷 accuracy 更像研究问题。

最需要收紧的地方有几个：

1. **Cross-flow context 要严格定义**

现在 $C_i = \{X_{i-K}, \dots, X_i, \dots, X_{i+K}\}$ 很自然，但如果包含未来 flow，在真实在线检测里会有 information leakage。建议明确区分：

- offline analysis 可以用前后窗口；
- online NIDS 应该只用 past-only context；
- “related flows” 是按时间邻近、same host pair、same source IP、5-tuple、还是 session/group。

2. **baseline 要更强**

你现在 baseline 有 LSTM、CNN+LSTM、Transformer、MLP，但 NIDS 里很强的实际 baseline 往往是 `Random Forest / XGBoost / LightGBM` on flow statistics。proposal 的风险部分也提到了它们，所以实验里最好一定加上，否则 reviewer 可能会问：“为什么不用最强的传统 flow baseline？”

3. **CIC-IDS2017 对 cross-flow 可能不够理想**

如果数据被打乱或 flow-level label 已经足够强，Stage-II context 可能提升不明显。建议做 per-attack analysis，重点看 scanning、DDoS、brute force、multi-stage/bursty attacks 这些 context/timing 更可能有帮助的类别。

4. **Suricata label bias 是真实风险**

你已经意识到了这一点，很好。建议把 claim 写得更谨慎：如果 internal labels 来自 Suricata，模型可能学到的是

“alert-consistent behavior”，不一定是真实 attack semantics。要靠 external labels、cross-dataset eval、unseen attack eval 来支撑泛化性。

5. **time encoding 需要具体化**

建议明确写成可实现版本，比如：

- packet order positional encoding;
- $\log_{1p}(\text{inter-arrival time})$;
- direction-aware timing;
- clipping/normalization strategy.

这样 Stage-I ablation 会更有说服力。

我建议下一步先把研究路线落成一个最小实验闭环：

1. `PCAP -> flows + packet sequences + labels`
2. strong flow baselines: RF/XGBoost/LightGBM/MLP
3. packet Transformer without time
4. packet Transformer + positional encoding
5. packet Transformer + inter-arrival-time encoding
6. past-only cross-flow context model
7. per-attack + latency/memory evaluation

小排版方面我渲染检查了第 2 和第 14 页，整体正常。第 14 页的 run-in headings 比如 “Limited Benefit of Packet-Level Modeling While...” 最好加冒号或句点，会更清楚。

总体评价：**idea 是可以做 thesis 的，而且研究问题站得住。关键是把 scope 控制好，先证明 Stage-I packet-time modeling，再谨慎推进 Stage-II cross-flow context。**

用户 2026-07-02 12:10:19

Files mentioned by the user:

custom-out.c: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/suricata-plugin/custom-out.c

My request for Codex:

我自定的suricata插件，我可以从PCAP中提取packet和flow相关的feature并生成csv文件。然后将此CSV中的数据输入到transformer等ML方法中

助手 2026-07-02 12:10:32

听起来这个插件就是你 thesis 里最关键的“数据桥”：把 PCAP 里的 packet/flow structure 保留下来，再交给 ML 模型。我先看一下 `custom-out.c` 的实际输出字段和组织方式，帮你判断它是否足够支撑后面的 Transformer 设计。

助手 2026-07-02 12:10:45

我看到你的插件已经不是简单 dump packet，而是同时维护了 per-flow aggregation，并且 packet CSV/flow CSV 分开输出，这个设计很适合你 proposal 里的 Stage-I / Stage-II。下面我再确认字段细节，看看有没有容易造成 leakage 或 sequence 对齐问题的点。

助手 2026-07-02 12:11:07

有几个设计点很强：你已经输出了 packet sequence 字段，也输出了 CICFlowMeter-style flow aggregation，所以同一批 CSV 可以同时做强 baseline 和你的 hierarchical model。现在我定位到几个训练时一定要处理的点，尤其是 label/identifier leakage。

助手 2026-07-02 12:14:46

可以，而且你的 `custom-out.c` 已经很贴合这个研究路线。

我看了代码：你现在有两个核心输出：

- Packet-level CSV:

[custom-out.c](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/suricata-plugin/custom-out.c:241) 包含 `flow\_id`, `timestamp\_us`, `direction`, `protocol`, `pkt\_len`, `payload\_len`, TCP flags, `flow\_iat\_us`, `direction\_iat\_us`, `packet\_label` 等，正好可以构造 Transformer 的 packet sequence `Xi`。

- Flow-level CSV:

[custom-out.c](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/suricata-plugin/custom-out.c:290) 包含 CICFlowMeter-style 的 flow statistics，比如 packet count、byte count、IAT stats、flag count、active/idle stats、最终 `label`，适合做 Random Forest / XGBoost / MLP / flow-level Transformer baseline。

推荐你的 ML pipeline 这样设计：

```
PCAP
-> Suricata custom plugin
-> stage1_packets.csv + stage1_flows.csv
-> group packets by flow_id
-> sort by timestamp_us / packet_id
-> build packet sequence Xi
-> merge flow label from flow CSV
-> train packet Transformer / time-aware Transformer / flow baselines
```

训练时要特别注意几个字段不要直接作为模型输入：

- `packet\_label` 和 flow CSV 的 `label` 只能作为 target，不能作为 feature。
- `flow\_id` 只能用于 grouping/join，不能作为 feature。
- `src\_ip`, `dst\_ip`, `src\_port`, `dst\_port` 要谨慎。它们可能造成 dataset-specific leakage；可以先不用，或者只用于构建 cross-flow context。
- 如果你的 label 来自 Suricata alert，论文里要明确这是 weak/rule-based label，避免 claim 成真正 ground truth。

最适合 Transformer 的 packet feature 可以先用这些：

```
direction, protocol, pkt_len, ip_len, payload_len,
ttl_or_hop_limit,
has_tcp, has_udp, has_icmp,
tcp_fin, tcp_syn, tcp_rst, tcp_psh, tcp_ack_flag, tcp_urg, tcp_ece, tcp_cwr,
tcp_window, tcp_header_len, udp_len,
icmp_type, icmp_code,
l7_proto,
log1p(flow_iat_us), log1p(direction_iat_us)
```

一句话：****你的插件已经能支撑 thesis 的核心 idea****。它让你既能做 flow-level baseline，又能保留 packet-level sequence 和 time-aware features，这是后面证明 “packet sequence modeling 是否比 flow aggregation 更有用” 的关键。

用户 2026-07-02 12:21:27

Files mentioned by the user:

comparison.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/comparison.py

baselines.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/baselines.py

utils.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/utils.py

metrics.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/metrics.py

export_embeddings.py:
C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/export_embeddings.py

dataset.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/dataset.py

splits.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/splits.py

preprocessing.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/preprocessing.py

build_mlp.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/build_mlp.py

config.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/config.py

data_io.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/data_io.py

trainer.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py

pipeline.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/pipeline.py

model.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/model.py

losses.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/losses.py

run_stage1.py: C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/run_stage1.py

My request for Codex:
这是我stage1的代码，仔细阅读并分析。

助手 2026-07-02 12:21:39

好的，我会把`s1/stage1`当成一个完整 pipeline 来读：先看入口和配置，再顺着 data loading、preprocessing、dataset、model、trainer、metrics、baseline/comparison 走一遍，最后给你结构、优点、风险和改进建议。

助手 2026-07-02 12:23:02

目录里有很多历史版本和实验配置，我会聚焦你这次点名的当前主线文件：`run\_stage1.py` 和 `stage1/\*.py` 这些无后缀版本。这样分析会更贴近你现在实际运行的 Stage 1。

助手 2026-07-02 12:23:13

入口和工具层我已经读完：主流程是`build\_data loaders -> build model -> train\_model -> optional export\_embeddings`。接下来我会看数据构建和预处理，因为这里最容易出现 label leakage、split leakage 或 sequence 顺序问题。

助手 2026-07-02 12:23:24

我已经确认预处理是 train-only fit，这一点很好；同时代码里也已经支持 random/chronological split。现在看 `pipeline.py`，它决定 split、precompute cache、sampler 和 DataLoader，这里会直接影响实验是否可复现。

助手 2026-07-02 12:23:35

模型部分已经读完：它实现了 A/B/C 三种 Stage-I 变体，包含 time-aware sinusoidal encoding、Transformer encoder、attention/mean/max pooling 和 flow fusion。现在我看训练与评估逻辑，重点检查 threshold、best model、loss 与 sampler 是否互相“叠太重”。

助手 2026-07-02 12:23:43

训练代码里我已经看到几处需要你后面整理的实验风险：sampler、class-balanced/focal loss、threshold tuning 同时启用时，结果解释会变复杂。最后我读 baseline/comparison，看看你的对照实验是否能支撑 proposal 的 RQ1。

助手 2026-07-02 12:23:58

对照实验文件很长，包含传统 ML、序列 baseline、消融和报告生成。我会再抓关键函数和行号，这样最终分析能精确指出哪里需要修。

助手 2026-07-02 12:24:05

我会做一个轻量的语法检查，不训练模型，只确认这些主文件能被 Python 编译。这样能区分“设计问题”和“会直接跑崩的问题”。

助手 2026-07-02 12:24:25

语法编译全部通过，说明没有直接的 syntax error。接下来我只做一次 import 检查，看看运行环境是否缺 `optuna/xgboost/lightgbm/scipy` 这类依赖。

助手 2026-07-02 12:24:43

import 检查在 bundled Python 里缺 `yaml`，所以这个环境不能直接运行 pipeline；这不代表你的项目环境也缺，`requirements.txt` 里可能已经有。为了理解实验实际输入，我再看默认 config 和 requirements。

助手 2026-07-02 12:25:06

默认 config 也看完了：目前是 chronological split、方案 C (packet encoder + separate flow fusion)、500 epoch、monitor `f1\_label1`。我现在整理几个关键行号，方便你之后直接定位修改。

助手 2026-07-02 12:25:38

我还会确认 sampler 和入口 debug 输出的位置。它们不是大问题，但会影响论文实验的可复现性和日志干净程度。

助手 2026-07-02 12:27:06

我仔细读完了主线代码。整体上，Stage1 已经是一个相当完整的 intra-flow 实验框架：从 CSV 清洗、flow-level split、train-only preprocessing、precomputed tensors、packet Transformer、flow fusion、threshold search、embedding export 到 baseline comparison 都有了。它能很好支撑你的 thesis 里 RQ1：packet-sequence/time-aware modeling 是否比 flow aggregation 更有信息量。

****核心结构****

你的主流程是：

```
`run_stage1.py -> pipeline.build_dataloaders -> Stage1Preprocessor -> precomputed npz ->
Stage1TimeAwareTransformer -> trainer.train_model -> export_embeddings`
```

模型支持三种方案：

- A: flow features 拼到每个 packet 上。
- B: 只用 packet sequence。
- C: packet Transformer 后再和 flow features 分层融合。默认 config 现在是 C。

这和你的 proposal 很贴：Stage1 学 intra-flow embedding，之后 export `z` 给 Stage2 做 inter-flow context。

****做得好的地方****

- [preprocessing.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/preprocessing.py:172) 只在 train split 上 fit scaler/OHE/clipping，避免了明显的数据泄漏。
- [splits.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/splits.py:167) 支持 chronological split，这对 PCAP/真实流量比 random split 更可信。
- [data_io.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/data_io.py:196) 按 `flow\_id` 和 `timestamp\_us` 排序构造 packet sequence，符合你的研究设定。
- [model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/model.py:117) 的 time-aware encoding 思路明确：`flow\_id` -> log1p -> expm1 -> cumulative time -> log，能表达不规则时间间隔。
- [export_embeddings.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/export_embeddings.py:1) 已经为 Stage2 保存 `flow\_id / label / z / timestamp / source / destination`，这一步非常关键。

****需要优先修的风险****

1. Precomputed cache 容易复用过期 tensor

[data_io.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/data_io.py:367) 只用 `seqLen + strategy + prefix + method` 判断缓存是否存在。如果你改了 feature list、scaler、split seed、CSV、flow fusion 配置，它仍可能直接复用旧 `npz`。这会导致实验结果和当前 config 不匹配。建议加 `force\_rebuild`，或把 config/data hash 写进文件名或 metadata。

2. Optuna 训练函数目前会直接报错

`\_train\_one\_epoch` 需要 `scaler` 参数，但

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py:499) 调用时没有传。主 `train\_model` 没问题，但 `train\_model\_for\_optuna` 会 TypeError。

3. comparison 里的传统 ML baseline 有 test threshold leakage

[comparison.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/comparison.py:600) 在 `evaluate\_ml\_model` 里用当前 loader 的 `y` 搜 threshold。如果这里传的是 test set，就等于用 test label 调阈值。应该在 val 上找 threshold，再固定到 test。

4. `find\_optimal\_threshold` 的约束逻辑被忽略

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py:740) 会找满足 precision/recall 约束的 `best`，但最后

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py:760) 又返回全局 class-1 F1 最优阈值。也就是说 `target\_precision/target\_recall` 实际不生效。

5. `run\_stage1.py` 的 debug 会消费 train sampler

[run_stage1.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/run_stage1.py:165)

在正式训练前遍历了 `loaders["train"]`。如果启用 WeightedRandomSampler, 这会推进 sampler generator 状态, 影响训练顺序。正式实验建议删掉或只看 `trainNoSampler`。

****研究设计建议****

默认 config 是方案 C, 也就是 packet Transformer + full flow features。这个模型可能很强, 但对 RQ1 来说不够“干净”, 因为提升可能来自 flow aggregation, 而不是 packet-time modeling。

建议论文里把 Stage1 主实验拆成:

- Flow-only strong baselines: RF / XGBoost / LightGBM / MLP。
- Packet-only B: no encoding。
- Packet-only B + positional encoding。
- Packet-only B + time encoding。
- Packet-only B + both。
- C: packet embedding + flow feature fusion, 作为增强模型。

这样你能更清楚地回答: “packet sequence 和 time-aware encoding 本身有没有贡献?”

我还做了语法检查: 这些主文件 `py\_compile` 全部通过。import 检查在 bundled Python 里因为缺 `yaml` 失败, 但你的 `requirements.txt` 里已经列了 `pyyaml`, 所以这是环境依赖问题, 不是代码语法问题。

用户 2026-07-02 13:08:42

方案B

下面是只有max_seq_length

```
sequence8: [TEST] Full metrics: {'accuracy': 0.982010731431799, 'macro_precision': 0.8836832121604303, 'macro_recall': 0.879236887852677, 'file': C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/B/attention_pooling_mild_cls2/stage1_config_0626_B_Both.yaml3, 'macro_f1': 0.8814457544489398, 'weighted_precision': 0.9819082755428861, 'weighted_recall': 0.982010731431799, 'weighted_f1': 0.9819583104766979, 'precision_label1': 0.7769784172661871, 'recall_label1': 0.767590618336887, 'f1_label1': 0.7722559885591705, 'auc': 0.9830632758458266, 'pr_auc': 0.8368694251226644, 'per_class_f1': {'0': 0.9906355203387089, '1': 0.7722559885591705}, 'confusion_matrix': [[33693, 310], [327, 1080]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.06234634028821646, 'threshold': 0.430000000000000027}
```

```
sequence16: [TEST] Full metrics: {'accuracy': 0.9819542502118046, 'macro_precision': 0.884991341352839, 'macro_recall': 0.8758008642617157, 'macro_f1': 0.8803348883599431, 'weighted_precision': 0.9817464683095168, 'weighted_recall': 0.9819542502118046, 'weighted_f1': 0.9818452533172927, 'precision_label1': 0.7798833819241983, 'recall_label1': 0.7604832977967306, 'f1_label1': 0.7700611730838431, 'auc': 0.9849720501061185, 'pr_auc': 0.8454853179059679, 'per_class_f1': {'0': 0.990608603636043, '1': 0.7700611730838431}, 'confusion_matrix': [[33701, 302], [337, 1070]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.06207364177607027, 'threshold': 0.47100000000000003}
```

```
sequence32: [TEST] Full metrics: {'accuracy': 0.9819824908218018, 'macro_precision': 0.8824893708595296, 'macro_recall': 0.880925490478379, 'macro_f1': 0.8817056626340283, 'weighted_precision': 0.9819458765725516, 'weighted_recall': 0.9819824908218018, 'weighted_f1': 0.9819640360960449, 'precision_label1': 0.7744468236973591, 'recall_label1': 0.7711442786069652, 'f1_label1': 0.7727920227920227, 'auc': 0.9854155913873647, 'pr_auc': 0.8571311582683947, 'per_class_f1': {'0': 0.9906193024760337, '1': 0.7727920227920227}, 'confusion_matrix': [[33687, 316], [322,
```



```
1085]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.058221365120060174, 'threshold': 0.401000000000000025}
```

```
sequence64: [TEST] Full metrics: {'accuracy': 0.9824060999717594, 'macro_precision': 0.8815947276477233, 'macro_recall': 0.8906845796310334, 'macro_f1': 0.8860805538653445, 'weighted_precision': 0.9826230585914077, 'weighted_recall': 0.9824060999717594, 'weighted_f1': 0.9825096408370192, 'precision_label1': 0.7718446601941747, 'recall_label1': 0.7910447761194029, 'f1_label1': 0.7813267813267813, 'auc': 0.9852139807639784, 'pr_auc': 0.8568879868326769, 'per_class_f1': {'0': 0.9908343264039076, '1': 0.7813267813267813}, 'confusion_matrix': [[33674, 329], [294, 1113]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05798349744683999, 'threshold': 0.391000000000000024}
```

```
sequence128: [TEST] Full metrics: {'accuracy': 0.9829991527817001, 'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1': 0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall': 0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607, 'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804, 'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606}, 'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05759464621893551, 'threshold': 0.45500000000000003}
```

```
max_seq_len=64 strategy=head_tail
"test_metrics": {
  "accuracy": 0.9822366563117764,
  "macro_precision": 0.8835218290969515,
  "macro_recall": 0.883783123279331,
  "macro_f1": 0.8836524270822737,
  "weighted_precision": 0.9822427135509747,
  "weighted_recall": 0.9822366563117764,
  "weighted_f1": 0.982239680831771,
  "precision_label1": 0.7762784090909091,
  "recall_label1": 0.7768301350390903,
  "f1_label1": 0.7765541740674956,
  "auc": 0.9856986363572042,
  "pr_auc": 0.8570448100368075,
  "per_class_f1": {
    "0": 0.9907506800970517,
    "1": 0.7765541740674956
  },
  "confusion_matrix": [
    [
      33688,
      315
    ],
    [
      314,
      1093
    ]
  ],
  "num_classes": 2,
```

```

"num_samples": 35410,
"loss": 0.05645895877765619,
"threshold": 0.46500000000000003
}

```

这是方案B跑出来的结果，详细解析。

结合我现有代码，我需要做什么才能使class1的f1,recall,precesion提高到0.83。

助手 2026-07-02 13:09:02

我先把你这组结果当成一个小型 ablation 来看：不同 `max\_seq\_len` 基本只差序列长度，核心问题是 class1 的 FP/FN trade-off。然后我会结合你当前 trainer、threshold、sampler、loss 和方案 B 的代码，给你一套最可能把 class1 拉到 0.83 附近的实验顺序。

助手 2026-07-02 13:09:40

我读了你这次方案 B 的 config：它确实是 pure packet 模式，`flow\_fusion.enabled=false`，`use\_weighted\_sampler=false`，`loss=focal`，`alpha\_mode=mild`，`monitor\_metric=pr\_auc`。这解释了结果为什么 AUC/PR-AUC 很高，但最终 class1 precision/recall/F1 还卡在 0.77-0.79：模型排序能力不错，决策边界和少数类分离还不够。

助手 2026-07-02 13:09:41

我会把你贴的 confusion matrix 转成 class1 的 FP/FN/TP 差距，看看“到 0.83”到底需要模型多做对多少条流。这个量化很重要，不然调参容易像摸黑拧旋钮。

助手 2026-07-02 13:11:12

你的方案 B 结果其实已经很稳定了：**纯 packet sequence，不用 flow feature，class1 F1 大约卡在 0.77-0.785**。这说明 Stage1 packet Transformer 确实学到了东西，但现在还没有把恶意流和正常流分得足够开。

****先看结果****

class1 真实样本数固定是 `1407`，正常类是 `34003`，正类比例只有约 `3.97%`，所以 accuracy 98% 不能作为主要指标。

当前最好的是：

```

max_seq_len=128, head
precision_label1 = 0.7881
recall_label1    = 0.7825
f1_label1        = 0.7853
TP=1101, FP=296, FN=306

```

如果你想让 **precision、recall、F1 都到 0.83**，大概需要：

```

TP >= 1168
FP <= 239
FN <= 239

```

也就是说，相比现在最好的 `seq128-head`，你需要同时做到：

```

67      flow
57      flow

```

这不是单纯调 threshold 就一定能做到的，因为降低 threshold 会提高 recall 但增加 FP，提高 threshold 会提高 precision 但增加 FN。你需要先看你代码里已经打印的：

```
VAL ORACLE
TEST AT VAL THRESHOLD
TEST ORACLE - DIAGNOSTIC ONLY
```

如果 `TEST ORACLE` 的 `f1\_label1` 也低于 0.83，说明 **threshold 调不出来，必须提升模型/特征/训练**。如果 `TEST ORACLE >= 0.83`，但 `TEST AT VAL THRESHOLD` 低，说明主要是 **threshold transfer / calibration 问题**。

****你的长度实验结论****

- `seq8 -> seq128`，F1 从 `0.7723` 提到 `0.7853`，说明更长 packet sequence 有帮助。
- `seq64-head` recall 最高：`0.7910`。
- `seq128-head` precision 和 F1 最好：`0.7881 / 0.7853`。
- `seq64-head\_tail` 没超过 `seq64-head`，但不能说明 head-tail 没用，因为你还没跑 `128/256 head\_tail`。

下一步建议先跑：

```
sequence:
  max_seq_len: 128
  strategy: head_tail
```

然后：

```
sequence:
  max_seq_len: 256
  strategy: head

sequence:
  max_seq_len: 256
  strategy: head_tail
```

****代码层面最该先改的点****

你现在的 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py:740)

虽然会找满足 precision/recall 约束的 threshold，但最后

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py:760) 又返回了 class1 F1 最高的 threshold，前面的约束实际没生效。

所以如果你想强制找 `precision>=0.83` 和 `recall>=0.83`，要先修这个 threshold selection。否则你在 config 里打开：

```
min_precision_class1: 0.83
min_recall_class1: 0.83
```

也不会真正控制最终 threshold。

****最推荐的实验顺序****

1. 先保留方案 B，跑 `128/256 + head/head\_tail`。
2. 加强 packet time feature:

```
features:
  packet:
    numerical:
      - pkt_len
      - ip_len
      - payload_len
      - ttl_or_hop_limit
      - tcp_window
      - tcp_header_len
      - udp_len
      - log_flow_iat_us
```

你现在 `log\_flow\_iat\_us` 是注释掉的。time encoding 用了 IAT，但把 log IAT 作为普通 packet feature 也可能帮分类头更直接使用时间强度。

3. 加回 `l7\_proto` 做一次 ablation:

```
categorical:
- ip_version
- protocol
- l7_proto
- icmp_type
- icmp_code
```

这个可能明显提升，但论文里要说明它可能带来 dataset-specific bias。

4. 尝试温和 oversampling，不要直接 50/50:

```
use_weighted_sampler: true
sampler_pos_fraction: 0.10
```

再试 `0.15`。你现在 `use\_weighted\_sampler=false`。

5. loss 换成 class-balanced focal:

```
loss: cb_focal
cb_beta: 0.9999
focal_gamma: 1.0
label_smoothing: 0.0
alpha_mode: none
```

6. 如果 recall 上来了但 FP 太多，再试 asymmetric:

```
loss: asymmetric
asl_gamma_pos: 0.0
asl_gamma_neg: 2.0
asl_alpha_pos: 0.60
```

还有一个很重要的小坑：你改 feature/config 后，要删掉对应 `precomputed/\*.npz`，因为 [data_io.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/data_io.py:367) 的缓存文件名没有包含 feature/config hash。否则你以为换了特征，实际模型还在吃旧 tensor。

我的判断：**方案 B 单靠现在这些 packet header features，要稳定到 P/R/F1 全部 0.83 会比较难，但不是完全没机会。最有希望的组合是 `seq\_len=256 + log\_flow\_iat\_us + l7\_proto + cb\_focal/controlled sampler + 修正 threshold 约束`。**如果目标是最终系统性能而不是纯 B ablation，那方案 C 或 Stage2 cross-flow context 更可能把 class1 F1 推过 0.83。

用户 2026-07-02 15:21:25

什么情况下方案 C 一定可以使 class 1 f1 达到0.83?

助手 2026-07-02 15:21:55

不能说“方案 C 一定可以”。严格讲，**没有任何模型结构能保证 class1 F1 到 0.83**。方案 C 只有在下面这些条件满足时，才很有机会把 F1 推到 0.83。

方案 C 会有效的条件：

1. **flow-level features 和 class1 有额外可分信息**

也就是说，B 中错分的那些 flow，必须能被 flow statistics 区分出来，比如：

`flow\_duration`, `packet\_count`, `flow\_iat\_\*`, `fwd/bwd\_iat\_\*`, `flag\_count`, `active/idle`, `bytes/s`, `packets/s` 等。

如果 B 已经把 packet sequence 中可见的信息都学完了，而 flow features 只是重复信息，C 提升会很小。

2. ****B 的 FN/FP 中, flow features 能同时减少一部分****

你目前最佳 B 大概是:

`TP=1101, FP=296, FN=306, F1=0.7853`

要到 F1=0.83, 一个比较均衡的目标是:

`TP≈1168, FP≤239, FN≤239`

也就是 C 至少要做到:

- 多找回约 `67` 个恶意 flow;
- 同时少误报约 `57` 个正常 flow。

如果 C 只提高 recall 但 FP 大涨, F1 不一定上去。

3. ****flow features 不能造成 train/test 分布偏差****

如果 flow-level 特征在 train/val/test 的分布一致, C 更容易提升。

如果 test 是未来时间段, 某些 flow statistics 发生 drift, C 可能反而过拟合。

4. ****C 的 fusion 方式要保护 packet branch****

你现在的 C 如果直接 gated/concat, 有可能 flow branch 把 B 里已经学好的 packet representation 干扰掉。更推荐你之前代码里的 `residual\_gated`:

```
flow_fusion:
  enabled: true
  inject_to_packets: false
  method: residual_gated
  residual_scale_init: -4.0
```

这个更适合从 B 出发做增强, 因为初始时更像 packet-only, 只有 flow feature 确实有用时才加进去。

5. ****必须和 B 用同一 split、同一 threshold 规则比较****

否则 C 到 0.83 可能只是 split 或 threshold tuning 的结果, 不是模型真的更强。

最现实的判断标准是: 先跑一个 ****flow-only strong baseline****。如果 RF/XGBoost/LightGBM 只用 flow features 已经能达到或接近 `class1 F1 >= 0.83`, 那方案 C 很有希望。如果 flow-only 也只有 0.75-0.78, 那么 C 不太可能“稳定保证”到 0.83。

一句话: ****方案 C 能到 0.83 的前提是 flow-level statistics 对 B 的错误样本有互补纠错能力, 而且 fusion 不能破坏 packet encoder。 **** 最先试 `residual\_gated C + seq128/256 + cb\_focal`。

用户 2026-07-02 15:32:11

mlp | P1=0.6346 | R1=0.6901 | F1_1=0.6612 | PR_AUC=0.7418 | AUC=0.9745 | threshold=0.5010
xgb | P1=0.6008 | R1=0.7562 | F1_1=0.6696 | PR_AUC=0.7319 | AUC=0.9781 | threshold=0.8070
lgbm | P1=0.6538 | R1=0.6887 | F1_1=0.6708 | PR_AUC=0.7332 | AUC=0.9745 | threshold=0.7680
这是baseline跑出来的结果。结合这个详细解析, 方案 C 是否一定可以使 class 1 f1 达到0.83?

助手 2026-07-02 15:32:38

不一定。结合这个 baseline, ****方案 C 更不能被认为“一定到 0.83” ****。

你的 flow-only baseline 结果是:

```
MLP   F1_1=0.6612, PR_AUC=0.7418
XGB    F1_1=0.6696, PR_AUC=0.7319
LGBM   F1_1=0.6708, PR_AUC=0.7332
```

而方案 B 最好大概是:

```
B seq128-head:
P1=0.7881
R1=0.7825
F1_1=0.7853
PR_AUC=0.8564
AUC=0.9860
```

这说明: **packet sequence branch 明显强于 flow feature branch**。flow statistics 单独只能到 `0.67`，不是一个接近 `0.83` 的强补充信号。

所以方案 C 的判断应该是:

```
C = B   packet representation + flow feature branch
```

如果 flow branch 只是弱信号, C 可能:

- 小幅提升, 比如 `0.785 -> 0.79/0.80`
- 没变化
- 甚至下降, 因为 flow branch 会引入噪声和 false positives

要到 `0.83`, 你现在的 B 需要大概做到:

```
TP: 1101 ->   1168       67   class1
FP: 296  ->   239       57   class0
```

而 flow-only baseline 的 precision 只有 `0.60-0.65`, 说明它本身误报不少。它不一定能帮你同时 “增加 TP + 减少 FP”。

什么情况下 C 可能有用?

只有一种情况: flow-only 虽然整体 F1 低, 但它抓到的是 **B 没抓到的那批恶意 flow**, 并且能识别一部分 B 的 false positives。

你需要做一个 error-overlap 分析:

```
B   FN       XGB/LGBM
B   FP       XGB/LGBM
```

如果 flow baseline 能覆盖很多 B 的错误, C 有希望。

如果 B 和 flow baseline 错的是同一批样本, C 基本不会到 0.83。

我的判断: 基于你现在的 baseline, **方案 C 不一定能到 0.83, 而且单靠 C 到 0.83 的概率不高**。更合理的预期是:

```
B: 0.785
C: 0.79 - 0.81
C   0.83
```

最推荐你下一步不是盲跑 C, 而是先做:

1. B vs flow-only error overlap 分析。
2. C 用 `residual\_gated`, 不要先用普通 concat/gated。
3. 跑 `seq128/256 + C residual\_gated + cb\_focal`。
4. 如果 C 仍不到 0.83, 把目标交给 Stage2 cross-flow context, 而不是强压 Stage1。

用户 2026-07-02 15:35:08

如果使方案A呢? 能否到达使 class 1 f1 达到0.83?

助手 2026-07-02 15:35:37

不一定。****方案 A 也不能保证 class1 F1 到 0.83****，而且从你现在的 baseline 看，A 到 0.83 的概率也不是特别高，但它值得跑。

方案 A 是：

```
packet token = [packet features ; full-flow statistical features]
packet Transformer
```

它和方案 C 的区别是：

```
A: flow features token-level packet flow
C: packet sequence z flow features flow-level
```

****结合你的结果判断****

目前：

```
B :
F1_1 ≈ 0.785

Flow-only baseline:
MLP F1_1=0.661
XGB F1_1=0.670
LGBM F1_1=0.671
```

这说明 flow features 单独并不强。方案 A 能否提升，取决于 flow features 是否能帮助 B 修正错误，而不是它本身有多强。

要到 0.83，你还是需要大约：

```
TP 60-70
FP 50-60
```

如果 flow features 不能帮你同时做到这两件事，A 不会到 0.83。

****方案 A 可能比 B 好的情况****

- 某些攻击不只靠 packet header pattern，而靠全局 flow 统计，比如 duration、packet count、IAT、bytes/s、active/idle。
- B 的 FN 里面，有一批 flow 在 flow-level 特征上很明显。
- flow statistics 和 packet token 之间有交互关系，比如“同样的 SYN pattern，在不同 duration/packet-rate 下含义不同”。

****方案 A 可能不如 C 的情况****

- flow features 单独较弱，重复拼到每个 packet 上会变成噪声。
- Transformer 可能学 flow-only shortcut，反而忽略 packet sequence。
- full-flow features 是完整 flow 结束后才知道的，如果你想做在线检测，A 会更容易被质疑“用了未来信息”。

我的预期比较现实：

```
B: 0.785
A: 0.78 - 0.81
A 0.83:
```

****如果要跑 A，我建议这样跑****

先用最强 B 设置作为基础，只改 flow fusion：

```
features:
  flow_fusion:
    enabled: true
    inject_to_packets: true
    method: concat
```

优先组合：

```
sequence:
```

```
max_seq_len: 128
strategy: head
```

然后试:

```
sequence:
  max_seq_len: 256
  strategy: head
```

训练建议:

```
training:
  loss: cb_focal
  cb_beta: 0.9999
  focal_gamma: 1.0
  label_smoothing: 0.0
  monitor_metric: pr_auc
  use_weighted_sampler: false
```

如果 recall 不够, 再试:

```
use_weighted_sampler: true
sampler_pos_fraction: 0.10
```

结论

方案 A 不是 “必然到 0.83”。根据你 flow-only baseline 只有 0.67, A 更像是对 B 的辅助增强, 而不是决定性突破。真正要判断它有没有希望, 需要看: A 是否能把 B 的 FN 减少, 同时不增加 FP。最值得跑的是 `A + seq128/256 + cb\_focal`, 但如果目标是稳定超过 0.83, 我会更看好 **C residual_gated** 或后面的 **Stage2 cross-flow context**。

用户 2026-07-02 15:41:41

我本来的预期是:

stage1-方案B class1的发f1=0.78

然后加上flow feature, stage1-方案C class1的发f1=0.80

然后将stage1生成的embedding输入到stage2, 然后获取context信息后, class1的发f1=0.85

这样的完美路线, 然后输出报告。但是现在看来无法实现了? 我该怎么办哪?

助手 2026-07-02 15:42:02

先别慌, 这条路线**没有死**, 只是需要把叙事从 “完美阶梯式提升” 改成更研究型、更稳的版本。

你原来的预期是:

B packet-only	F1_1 $\approx$ 0.78
C + flow features	F1_1 $\approx$ 0.80
Stage2 context	F1_1 $\approx$ 0.85

现在实际情况更像:

B packet-only	F1_1 $\approx$ 0.785	
flow-only baseline	F1_1 $\approx$ 0.67	flow feature
C + flow features		
Stage2 context		

所以不是 “无法实现”, 而是: **Stage1-C**

不一定能成为漂亮的中间台阶。这在论文里其实可以变成一个很好的发现: packet-level temporal representation 是主要信号, flow-level aggregation 单独较弱; 简单加入 flow statistics 并不必然提升, 需要更谨慎的融合或交给 Stage2 context 建模。

你现在最应该做的是把目标改成三层证据, 而不是硬追每一步都提升:

1. B packet-sequence modeling
2. C flow feature fusion
3. Stage2 cross-flow context

如果 C 只到 0.79，甚至没有超过 B，也不是失败。你可以写：

> Flow-level statistics alone were substantially weaker than packet-sequence modeling. Direct fusion provided limited additional gains, suggesting that coarse aggregated features may not add much information beyond packet-level temporal patterns. Therefore, the second stage focuses on contextual relationships across flows rather than simply injecting per-flow statistics.

这比强行说 C 一定提升更可信。

****你该怎么办****

先做这几件最关键的：

1. ****把 B 做扎实****

固定最强 B：`seq128/head` 或继续跑 `seq256/head`、`seq256/head\_tail`。B 已经是你的 strong Stage1 baseline。

2. ****跑 C，但把预期改成 ablation****

不要求 C 必须到 0.83。跑：

```
flow_fusion:
  enabled: true
  inject_to_packets: false
  method: residual_gated
```

如果 C > B，报告提升。

如果 C ≈ B 或低于 B，报告 “flow statistics are not strongly complementary at Stage1”。

3. ****做 error overlap 分析****

看 flow-only 是否能纠正 B 的 FN/FP。这个会告诉你 C 为什么没提升，而不是只报一个分数。

4. ****把主要希望放到 Stage2****

Stage2 的研究问题本来就是 cross-flow context。它不依赖 C 必须强。你可以用 B 的 embedding 作为 Stage2 输入：

```
Stage1-B embedding -> Stage2 context model
```

不一定要用 C embedding。

5. ****调整最终目标****

不要把 thesis 成败绑定到 `0.85`。更好的目标是：

```
Stage2 improves over Stage1-B on class1 recall/F1, especially for context-dependent attacks.
```

如果 Stage2 从 `0.785 -> 0.81/0.82`，这仍然是有价值的结果。到 `0.85` 是 bonus，不是生死线。

****最适合你的新论文路线****

我建议改成：

```
RQ1: Does packet-level time-aware modeling outperform flow-level aggregation?
Answer: Yes. B ≈ 0.785 vs flow-only ≈ 0.67.
```

```
RQ2: Do flow-level statistics add complementary information to packet embeddings?
Answer: Limited / conditional. C may improve slightly or not.
```

```
RQ3: Does inter-flow context improve detection beyond independent flow classification?
Answer: Stage2 experiment.
```

这样你的结果反而更清楚：****真正强的是 packet sequence，真正值得探索的是 cross-flow context，而不是简单 flow-stat fusion。****

你没有失败。你现在只是从“想象中的漂亮曲线”进入了“真实研究会出现的形状”。这其实更像一篇 thesis。

用户 2026-07-02 15:56:11

我该如何做这个分析？

B 的 FN 中，有多少被 XGB/LGBM 判对？

B 的 FP 中，有多少被 XGB/LGBM 判错回正常？

助手 2026-07-02 15:56:23

我先看一下你现在的 Stage1/benchmark 有没有已经保存 `flow\_id / y\_true / prob / pred`，这样这个重叠分析可以直接落到你现有文件结构上，而不是凭空给一段不贴合的代码。

助手 2026-07-02 15:58:28

我看到了一个关键点：现在的训练代码会保存

metrics、embedding、metadata，但不一定保存逐条样本的预测概率。所以这个分析最好做成一个“prediction table”：每个 `flow\_id` 一行，同时放 B、XGB、LGBM 的 `prob/pred/y\_true`。

助手 2026-07-02 15:58:44

你的 `flow\_only\_baseline.py` 已经在计算 `val\_prob/test\_prob`，只是现在主要保存汇总指标，没有把每个 flow 的概率保存出来。最小改法就是在这里多保存一个 CSV；B 这边也同理，从 best model 对 test loader 跑一遍并保存概率。

助手 2026-07-02 15:59:55

要做这个分析，核心不是看混淆矩阵，而是要保存“逐个 flow 的预测结果”。

你需要一张表，至少包含：

```
flow_id, y_true,
b_prob, b_pred,
xgb_prob, xgb_pred,
lgbm_prob, lgbm_pred
```

然后定义：

```
b_fn = (df.y_true == 1) & (df.b_pred == 0) # B
b_fp = (df.y_true == 0) & (df.b_pred == 1) # B

xgb_rescue_fn = b_fn & (df.xgb_pred == 1) # XGB B FN
xgb_fix_fp = b_fp & (df.xgb_pred == 0) # XGB B FP

lgbm_rescue_fn = b_fn & (df.lgbm_pred == 1)
lgbm_fix_fp = b_fp & (df.lgbm_pred == 0)
```

分析代码大概是这样：

```
import pandas as pd

df = pd.read_csv("merged_test_predictions.csv")

def report(model):
    b_fn = (df.y_true == 1) & (df.b_pred == 0)
    b_fp = (df.y_true == 0) & (df.b_pred == 1)

    rescued_fn = b_fn & (df[f"{model}_pred"] == 1)
```

```

fixed_fp = b_fp & (df[f"{model}_pred"] == 0)

harmed_tp = (df.y_true == 1) & (df.b_pred == 1) & (df[f"{model}_pred"] == 0)
harmed_tn = (df.y_true == 0) & (df.b_pred == 0) & (df[f"{model}_pred"] == 1)

print(f"\n[{model}]\n")
print(f"B FN total          : {b_fn.sum()}")
print(f"{model} rescued B FN   : {rescued_fn.sum()} ({rescued_fn.sum()/max(b_fn.sum(),1):.2%}")
print(f"B FP total          : {b_fp.sum()}")
print(f"{model} fixed B FP    : {fixed_fp.sum()} ({fixed_fp.sum()/max(b_fp.sum(),1):.2%}")
print(f"{model} harmed B TP   : {harmed_tp.sum()}")
print(f"{model} harmed B TN   : {harmed_tn.sum()}")

report("xgb")
report("lgbm")

```

你现在代码里已经有基础：`flow\_only\_baseline.py` 里 XGB/LGBM 已经算出了 `test\_prob`；B 模型的 `test\_loader` 里也有 `flow\_id` 和 `label`。缺的只是把这些逐条保存成 CSV。

解释结果时看这个判断：

- 如果 XGB/LGBM 能救很多 B 的 FN，比如 B seq128 有 `FN=306`，能救出 60-100 个，同时还能修正一部分 FP，那么说明 flow feature 对方案 C 有真实互补价值。
- 如果 XGB/LGBM 在 B 的 FN 里也大多判成 0，说明它救不了漏报，方案 C 很难明显提高 recall。
- 如果它能修正 B 的 FP，但同时把很多 B 的 TP 判成 0，那融合后可能 precision 提高、recall 下降，F1 不一定涨。
- 如果 `rescued\_fn + fixed\_fp` 很小，或者 `harmed\_tp + harmed\_tn` 很大，那方案 C 不应该被期待到 0.83。

对你现在最关键的判断是：B seq128 目前大约 `TP=1101, FP=296, FN=306`。想冲到 `F1≈0.83`，flow branch 至少要在 B 的错误里提供明显补救，粗略需要几十到上百个有效纠错，而且不能引入太多新错误。这个 overlap 分析就是为了验证“方案 C 有没有这个可能性”。

用户 2026-07-02 16:09:27

还有什么其他方案吗？可以加入 flow feature 之后，让他比 B 方案，性能更好？请提供所有可能的方案

助手 2026-07-02 16:09:43

我把思路按“最可能有效、最容易和你现有代码结合”的顺序整理。先说结论：加入 flow feature 想稳定超过 B，关键不是“把 flow 拼进去”，而是让 flow 分支只在它真的有互补信息时发力。由于你的 flow-only baseline 只有 $F1 \approx 0.67$ ，强行融合很容易拖累 B。

助手 2026-07-02 16:10:17

可以。有很多方案，但我建议你不要把它们都当成“必须跑完”，而是分优先级。你现在最值得尝试的是这些：

****最推荐****

1. ****方案 D：分数级融合 / Stacking****

不要直接把 flow feature 塞进 Transformer，而是先让 B 保持原样，然后训练一个小的 meta-classifier：

```

input = [B_prob, XGB_prob, LGBM_prob, maybe B_entropy]
output = final_pred

```

这个方案最适合你现在的情况，因为 B 很强，flow-only 较弱。meta-classifier 可以学习：

B			
XGB/LGBM	B	FN	FP

这比方案 C 的“盲目融合”更稳。

2. **方案 E: 只在 B 不确定区间使用 flow feature**

例如:

```
B_prob < 0.2 ->
B_prob > 0.8 ->
0.2 <= B_prob <= 0.8 -> XGB/LGBM/flow fusion
```

这样 flow feature 只处理 B 犹豫的样本，不干扰 B 已经判得很准的样本。这个很适合提升 F1，因为它减少了 flow 分支“帮倒忙”的机会。

3. **方案 F: flow-based veto / rescue rule**

根据 overlap analysis 设计规则:

```
Rescue FN:
  B_pred=0  B_prob          XGB/LGBM          ->

Fix FP:
  B_pred=1  B_prob          XGB/LGBM          ->
```

这不是最终论文主模型也没关系，它可以作为一个强 ablation，证明 flow feature 是否真的互补。

改进方案 C

4. **Residual-gated fusion**

你现有代码里最应该用这个，而不是普通 concat:

```
z_final = z_packet + gate * flow_projection
```

关键是 gate 要小，让模型默认相信 packet branch，只有必要时才引入 flow feature。

5. **Packet head + Flow head + Fusion head 多任务学习**

模型输出三个结果:

```
packet_logits
flow_logits
fused_logits
```

训练 loss:

```
loss = fused_loss + 0.3 * packet_loss + 0.1 * flow_loss
```

这样可以防止融合后 packet 表示被 flow feature 拉坏。这个对你尤其重要，因为 B 本身强，flow-only 弱。

6. **Confidence-aware gate**

让 gate 不只看 flow feature，也看 B 的不确定性:

```
gate = f(flow_feats, B_prob, entropy)
```

如果 B 很确定，gate 接近 0；如果 B 不确定，gate 变大。这个比普通 gated fusion 更合理。

7. **FiLM / Adapter 融合**

不用拼接 flow feature，而是让 flow feature 生成缩放和平移参数:

```
packet_hidden = gamma(flow) * packet_hidden + beta(flow)
```

这相当于让 flow feature 调节 packet representation，而不是替代它。通常比 concat 更稳定。

8. **Flow token**

把 flow feature 编码成一个特殊 token，放进 packet sequence:

[CLS], packet1, packet2, ..., packetN, [FLOW]

然后让 Transformer 自己学 packet-flow interaction。你项目里好像已经有`model\_flow\_token.py`，可以作为一个方向。

****改进 flow feature 本身****

9. ****只选对 B 错误有帮助的 flow feature****

不要把所有 flow feature 都塞进去。你应该专门分析：

```
B   FN vs B   TP   flow feature
B   FP vs B   TN   flow feature
```

然后只保留能区分这些错误的特征。否则弱 flow feature 会增加噪声。

10. ****加入方向性 flow feature****

比总量统计更有用的通常是：

```
fwd_pkt_count, bwd_pkt_count
fwd_bytes, bwd_bytes
fwd/bwd ratio
fwd_iat_mean, bwd_iat_mean
syn_count, ack_count, rst_count
bytes_per_second
packets_per_second
```

攻击流和正常流的方向结构可能比总长度、总包数更有区分度。

11. ****prefix-flow feature****

如果 B 只看前 64/128 个 packet，那么 flow feature 最好也只基于同样范围计算。否则 full-flow feature 和 packet sequence 的时间范围不一致，模型可能学到奇怪的关系。

****训练策略****

12. ****Hard-error focused fusion****

先训练 B，然后标出训练集里 B 的 FN/FP。训练 C 或 D 时提高这些 hard samples 的权重：

```
B       weight = 2 or 3
B       weight = 1
```

目的很明确：让 flow feature 专门学习修正 B 的错误。

13. ****冻结 B，再训练 flow fusion****

先加载 B 的 best model，冻结 packet Transformer，只训练 flow encoder + fusion head。稳定后再小学习率联合 fine-tune。

这比从头训练 C 更稳，因为它保护了 B 已经学到的 0.78 F1。

****我最建议你的实验路线****

按这个顺序做：

1. B + XGB/LGBM overlap analysis
2. D B_prob + XGB_prob + LGBM_prob stacking
3. E B uncertain zone flow model
4. C residual_gated B fusion
5. C packet head + flow head + fused head
6. Flow token / FiLM ablation

如果目标是“让加 flow feature 后比 B 更好”，最有希望的不是原始方案 C，而是：

```
B + flow score-level stacking
```


B uncertain-zone reranking
residual-gated C with frozen packet branch

这三个最符合你现在的实验现象。B 已经强，flow-only 弱，所以策略应该是“保护 B，然后让 flow feature 修错”，不是“让 flow feature 和 packet feature 平等竞争”。

用户 2026-07-03 08:32:07

mlp | P1=0.6346 | R1=0.6901 | F1_1=0.6612 | PR_AUC=0.7418 | AUC=0.9745 | threshold=0.5010
xgb | P1=0.6008 | R1=0.7562 | F1_1=0.6696 | PR_AUC=0.7319 | AUC=0.9781 | threshold=0.8070
lgbm | P1=0.6538 | R1=0.6887 | F1_1=0.6708 | PR_AUC=0.7332 | AUC=0.9745 | threshold=0.7680

这是baseline跑出来的结果。从这个结果来看，有没有可能我用suricata抽取的feature是错误的？我需要重新抽取吗？详细解析

助手 2026-07-03 08:32:22

我会从两个层面判断：第一，这个 baseline 结果本身说明什么；第二，什么现象才真的指向 Suricata feature 抽取错误。先别急着重抽，低 F1 不等于抽错，尤其你这里 AUC 很高，这个信号很关键。

助手 2026-07-03 08:32:35

还有一个容易被忽略的点：你的 flow-only baseline 是“只用聚合 flow 统计”，它天然会丢掉 packet sequence 的局部形态。所以它比 B 低很多是合理的，不自动说明 Suricata 插件错了。真正要查的是这些 feature 是否有异常值、单位错误、方向错误、标签/flow_id 对齐错误。

助手 2026-07-03 08:35:32

我顺手看了你本地 flow CSV

的前几行：字段结构是完整的，至少不是“明显少了一大片列”的情况。但第一批样本里有很多单包 flow、`duration=0`、bytes/s=0，这类在真实流量里可以出现，不过它会让 flow-only baseline 天然变弱，需要做分布检查而不是马上重抽。

助手 2026-07-03 08:36:48

有可能存在 feature 定义问题，但**从这组 baseline 结果本身看，不像是 Suricata feature 整体抽错了**。更准确的判断是：

flow feature	packet sequence
“	” “ ”

****为什么不像整体抽错****

如果 flow feature 完全错了、label 对不上、flow_id 对不上，通常会看到：

AUC	0.5
PR_AUC	0.04
F1	

但你现在是：

MLP	PR_AUC=0.7418, AUC=0.9745
XGB	PR_AUC=0.7319, AUC=0.9781
LGBM	PR_AUC=0.7332, AUC=0.9745

这说明 flow feature 对 label 是有明显排序能力的。尤其 PR_AUC 远高于正类比例，说明不是随机噪声。

真正的问题在于 class 1 的 precision 不够：

```
XGB: P1=0.6008, R1=0.7562
LGBM: P1=0.6538, R1=0.6887
```

也就是说，flow feature 能抓到不少攻击，但同时把很多正常 flow 也打成攻击。这个更像是：

```
flow-level
```

```
threshold
```

而不是“抽取代码全错”。

****一个值得重点检查的地方****

你插件里 flow 的 packet length 统计现在用的是 ****L4 payload length****，不是 IP total length。代码里这里写得很明确：[custom-out.c](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/suricata-plugin/custom-out.c:551)

```
static uint32_t GetCicPacketLen(const Packet *p)
{
    return (uint32_t)p->payload_len;
}
```

这不一定是 bug，但它会导致很多 TCP ACK、SYN、RST 等无 payload 的包在 flow length 统计里变成 0。你本地 flow CSV 前几行也能看到这种情况：有 TCP 包、header/window/flag 都有值，但 `total\_length\_of\_fwd\_packets=0`、`packet\_length\_mean=0`。

这会让 flow-only baseline 变弱，因为很多有用的包大小信息被压成了 0。B 方案反而更强，是因为 packet branch 里还用了：

```
pkt_len
ip_len
payload_len
tcp_window
tcp_header_len
ttl
flags
```

所以 B 没有完全丢掉这些信息。

因此这里的结论是：

```
          flow feature
payload-length    IP-total-length
```

例如新增：

```
total_fwd_ip_bytes
total_bwd_ip_bytes
fwd_ip_len_mean/std/max/min
bwd_ip_len_mean/std/max/min
packet_ip_len_mean/std/max/min
total_fwd_payload_bytes
total_bwd_payload_bytes
```

这样你就不用纠结“CIC 风格到底算 payload 还是 packet length”，两个都给模型，让模型自己选。

****什么时候才需要重新抽取****

我建议你不要现在立刻重抽。先做 sanity check。只有下面这些检查失败，才说明需要重抽：

1. ****flow_id 对不上****

检查 packet CSV groupby 后的 flow_id 是否都能在 flow CSV 找到。

```
packet flow_id    ≈ flow CSV
```

```
flow_id
flow_id
```

2. **packet 数量对不上**

对每个 flow:

```
packet_csv    flow    packet

flow_csv    total_fwd_packets + total_backward_packets
```

如果大量不一致，说明聚合逻辑或输出时机有问题。

3. **duration 异常**

单包 flow 的 `flow\_duration=0` 正常。

但如果很多多包 flow 也是：

```
total_packets > 1    flow_duration = 0
```

那就有问题，可能 timestamp、flow flush、聚合更新顺序不对。

4. **方向异常**

如果绝大多数 flow 都是：

```
total_backward_packets = 0
```

而你的 PCAP 明明包含双向通信，那方向判断或 flow 构建可能有问题。

5. **长度异常**

如果大量 flow 的：

```
packet_length_mean = 0
total_length_of_fwd_packets = 0
total_length_of_bwd_packets = 0
```

但 packet CSV 里的 `pkt\_len/ip\_len` 明明不为 0，那不是抽错，而是你当前 flow length 用的是 payload。这个情况下建议重抽“增强版 flow feature”。

6. **label 对不上**

检查：

```
flow_label == same_flow    packet_label    max
```

如果不一致很多，baseline 结果就不可信。

7. **precomputed cache 过期**

你的 flow-only baseline 不是直接读 raw CSV，而是读：

```
seqLen64headtrainC.npz
seqLen64headvalC.npz
seqLen64headtestC.npz
```

所以如果你改过 feature/config，但没有重新生成 `.npz`，baseline 可能用的是旧 feature。这个很重要。每次改 flow feature 后，要删除或重建 `precomputed`。

****我现在的判断****

不用马上推翻 Suricata 插件，也不用马上全量重抽。

更合理的结论是：

```
1.    flow feature
```

****我建议你下一步这样做****

```
payload-based flow stats
IP-length-based flow stats

flag ratio
first-N packet prefix flow stats
```

第 25 页

label 对不上

检查:

flow label == same flow 下 packet_label 的 max

请结合我的代码, 给我提供上面检查的代码

助手 2026-07-03 08:48:14

我来直接给你做成一个可运行的诊断脚本, 放在项目里。它会读 packet CSV 和 flow CSV, 输出每一项检查的计数、比例、是否通过, 并保存几份异常样本 CSV, 方便你判断要不要重抽。

助手 2026-07-03 08:48:35

我会把检查写成独立脚本, 不改你的训练流程。这样你可以对 train/test 任意一对 CSV 单独跑, 也可以在重新抽取后反复跑同一套 sanity checks。

助手 2026-07-03 08:48:54

脚本会尽量少读无关列: packet CSV 用 chunk 聚合, 所以即使 packet 文件比较大也能跑。输出会包含 JSON 总结、TXT 可读报告, 以及每类异常的样本 CSV。

助手 2026-07-03 08:51:03

脚本已经加好了。我现在先做语法检查, 确认它至少能直接运行; 如果通过, 我再给你一条按你当前数据文件可直接执行的命令。

助手 2026-07-03 08:51:09

本机 `python` 命令没有在这个 shell 里正常解析, 我换用 Codex bundled runtime 来做同样的语法检查。

助手 2026-07-03 08:51:29

语法检查通过了。现在我先看一下这对 CSV 的大小, 如果不是太离谱, 可以顺手跑一次完整检查, 给你看实际会得到什么。

助手 2026-07-03 08:51:41

packet CSV 大约 191MB, flow CSV 大约 18MB, 可以跑。这个检查会生成一些诊断文件到 `s1/diagnostics/...`, 不会改你的原始数据。

助手 2026-07-03 08:53:41

我已经给你加了一个独立检查脚本:

```
[check_suricata_csv_consistency.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/check_suricata_csv_consistency.py:1)
```

可以这样运行:

```
python s1/check_suricata_csv_consistency.py `
--packet_csv s1/data/firewall_test_20250731_160700-stage1_packets.csv `
--flow_csv s1/data/firewall_test_20250731_160700-stage1_flows.csv `
```

```
--out_dir s1/diagnostics/firewall_test_suricata_checks
```

如果你本机 `python` 命令不可用，用这个：

```
C:\Users\XiaoyanXiong\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe s1/
check_suricata_csv_consistency.py --packet_csv s1/data/firewall_test_20250731_160700-stage1_packets.csv --
flow_csv s1/data/firewall_test_20250731_160700-stage1_flows.csv --out_dir s1/diagnostics/
firewall_test_suricata_checks
```

我已经对你当前这对 CSV 跑了一次，结果很关键：

```
[PASS] duplicate flow_id in flow CSV: 0/38671
[PASS] packet flow_id missing in flow CSV: 1/38672
[PASS] flow CSV flow_id missing in packet CSV: 0/38671
[PASS] packet count mismatch: 0/38671
[PASS] direction count mismatch: 0/38671
[PASS] invalid packet direction values: 0/38671
[PASS] multi-packet flow_duration == 0: 12/30610
[PASS] flow duration disagrees with packet timestamps: 0/30610
[PASS] flows with total_backward_packets == 0: 15958/38671
[PASS] flow length stats zero but packet length positive: 9963/38671
[PASS] flow label != max(packet_label): 0/38671
```

结论：**从这些 sanity checks 看，你现在的 Suricata 抽取逻辑没有明显错误，不需要因为“抽错了”而重抽。**

唯一值得注意的是：

```
flow length stats zero but packet length positive: 9963/38671, 25.8%
```

这和我们之前判断一致：你的 flow length 统计主要是 payload-based，所以很多 ACK/SYN/RST 这类无 payload 包会让 flow length 统计为 0。但这不是 bug，更像是 feature definition 不够丰富。后续如果要提升 C 方案，建议加一版 enhanced flow feature：同时保留 `payload\_len` 统计和 `ip\_len/pkt\_len` 统计。

输出文件在这里：

```
s1/diagnostics/firewall_test_suricata_checks/
```

里面有 summary JSON、TXT report、聚合后的 packet-flow 表，以及各类异常样本 CSV。

用户 2026-07-03 08:57:09

我现在的baseline用了所有的flow csv中的feature，有没有可能只有部分有用呢？能否给我提供代码判断那些属性有用？然后重新再跑一次baseline，看是否有改善？

助手 2026-07-03 08:57:25

可以，而且这一步很值得做：flow-only baseline 用“所有 flow feature”时，弱特征、冗余特征、甚至噪声特征都可能拉低 precision。我要先看一下你 `.npz` 里有没有保存 `flow\_feature\_names`，这样筛选结果能直接告诉你具体是哪几列有用。

助手 2026-07-03 09:03:53

我看到当前 `.npz` 很可能只保存了 `flow\_feats` 和 `labels`，未必保存 feature names。为了让结果可解释，我会让脚本支持两种模式：优先从 `stage1\_preprocessor.joblib` 读取真实列名；如果读不到，就用 `flow\_feat\_000` 这种编号，不影响筛选和重跑。

助手 2026-07-03 09:04:04

本地没有找到 C 目录下的 `stage1\_preprocessor.joblib` 和 `seqLen64headtrainC.npz`，所以我会写得更通用：你在 Colab 或本机把实际 C 的 train/val/test npz

传进去即可；如果有 `preprocessor.joblib`，就能输出真实列名。

助手 2026-07-03 09:04:53

我准备加一个新脚本：它先在 `train` 上计算 `feature ranking`，再用 `val` 选择 `top-K`，最后在 `test` 上做一次合法评估；同时会输出“筛选后的 `npz`”，你可以把它直接喂给你现有的 ``flow_only_baseline.py`` 重跑 `MLP/XGB/LGBM`。这样 `test` 不参与选特征，实验更干净。

助手 2026-07-03 09:07:14

脚本已经写好。我注意到一个小地方：`LightGBM` 训练这里不需要 ``eval_set``，否则还要把 ``y_val`` 传进去，容易引入混乱；我会顺手把它简化掉再做语法检查。

助手 2026-07-03 09:07:26

接下来我检查脚本语法，并顺便确认当前 `workspace` 里有没有可直接用于演示的 `C`npz``。如果没有，我会给你按 `Colab/本机路径` 替换即可运行的命令。

助手 2026-07-03 09:10:09

语法检查通过了。本地这个 `workspace` 目前没有找到 `C` 的 ``trainC/valC/testC.npz``，所以我不能在这里直接重跑你的 `C flow baseline`；但这个脚本已经可以在你有 `C`npz`` 的位置运行，尤其是你 `notebook` 里 ``/content/drive/.../precomputed/seqLen64headtrainC.npz`` 那组。

助手 2026-07-03 09:10:33

可以，而且我已经给你加好了脚本：

```
[flow_feature_selection.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/flow_feature_selection.py:1)
```

它会做三件事：

1. 计算每个 `flow feature` 的重要性：``f_classif``、``mutual_info``、`XGB/LGBM/ExtraTrees importance`。
2. 按 `top-K` 特征重新训练 `XGB/LGBM/ExtraTrees`，在 ``val`` 上选阈值，再到 ``test`` 上评估。
3. 自动保存筛选后的 ``npz``，你可以直接用原来的 ``flow_only_baseline.py`` 重新跑 ``mlp xgb lgbm``。

用法示例：

```
python s1/flow_feature_selection.py `
--train_npz path/to/seqLen64headtrainC.npz `
--val_npz path/to/seqLen64headvalC.npz `
--test_npz path/to/seqLen64headtestC.npz `
--out_dir s1/flow_feature_selection_results `
--save_selected_npz `
--topk 5,10,15,20,30,40,50,all `
--selection_metric val_f1_label1
```

如果你有对应的 ``stage1_preprocessor.joblib``，加上这个参数可以输出真实 `feature` 名字：

```
--preprocessor_joblib path/to/stage1_preprocessor.joblib
```

输出会在：

```
s1/flow_feature_selection_results/
```

重点看这几个文件：


```

flow_feature_importance_ranked.csv      # flow feature
flow_feature_subset_eval.csv           # top-K          val/test
selected_features.csv                  #
selected_npz_topK_xxx/                 # train/val/test npz

```

然后用筛选后的`.npz`重新跑你的 baseline:

```

python s1/flow_only_baseline.py `
--train_npz s1/flow_feature_selection_results/selected_npz_top*/train_selected_flow_feats.npz `
--val_npz s1/flow_feature_selection_results/selected_npz_top*/val_selected_flow_feats.npz `
--test_npz s1/flow_feature_selection_results/selected_npz_top*/test_selected_flow_feats.npz `
--out_dir s1/flow_feature_selection_results/baseline_on_selected_features `
--models mlp xgb lgbm

```

注意: 正式报告里应该用`--selection\_metric val\_f1\_label1`或`val\_pr\_auc`, 不要用`test\_f1\_label1`选特征。`test\_\*`只能做诊断, 否则会 test leakage。

我这里已经做了语法检查, 脚本通过。但本地 workspace 里没有找到你的 C 方案`seqLen64headtrainC.npz / valC / testC`, 所以我没法直接替你跑完整 baseline。你在 Colab 或有这些 C`.npz`的机器上运行上面的命令即可。

判断结果时:

selected top-K flow feature	all features feature
val test feature selection	val
top-K all flow-only	flow
top-K test selected flow feature	C / stacking

用户 2026-07-03 09:22:40

我现在的baseline用了所有的flow csv中的feature, 有没有可能只有部分有用呢? 能否给我提供代码判断那些属性有用? 然后重新再跑一次baseline, 看是否有改善? 使用ar002_et12_20260511_002-stage1_flows.csv文件来做。因为.npz文件中没有属性名, 很难判断

助手 2026-07-03 09:22:57

可以, 而且这一步很应该做: flow-only baseline 用“所有 flow feature”并不一定最好, 弱特征、重复特征、payload-length 为 0 的特征、甚至泄漏型/ID型特征都可能拉低模型。我要给你做一个直接读原始 flow CSV 的脚本, 保留属性名, 先做 feature ranking, 再用 top-K 特征重跑 baseline。

助手 2026-07-03 09:23:20

我找到了你要用的文件: `C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\ar002\_et12\_20260511\_002-stage1\_flows.csv`, 大小约 83MB。接下来我会写一个新脚本, 不依赖`.npz`, 直接从这个 CSV 做: 清洗、split、feature ranking、top-K baseline。

助手 2026-07-03 09:23:36

这里我会做两种排名, 避免只相信一种方法: 一个是 univariate ranking (每个特征单独和 label 的关系), 另一个是模型重要性 ranking (ExtraTrees / XGBoost / LightGBM 如果可用)。然后用 top

10/20/30/40/60/all 重新训练，比较 test F1/PR-AUC。

助手 2026-07-03 09:23:49

你的 C 配置里默认 flow numerical 基本是 CICFlowMeter 风格的统计列，binary 还包括 init-window 是否存在这类列。这类特征里确实会有很多强相关/弱贡献/近似常数列，所以做 top-K 选择是合理的。

助手 2026-07-03 09:24:52

我还会让脚本输出“去相关后的 top-K”。原因是 flow feature 里很多列高度重复，比如 `packet\_length\_std` 和 `packet\_length\_variance`、各种 total/mean/rate。直接选 top 20 可能选到一堆同义特征，去相关后更容易看到真正有用的属性组。

助手 2026-07-03 09:27:38

脚本已写好。现在我先做语法检查，然后用本地数据跑一个轻量版本（先跑 logreg/ExtraTrees），确认 ranking 和 top-K baseline 能正常产出；XGB/LGBM 如果本地没装，脚本会自动跳过，你在 Colab/服务器上可以照样跑。

助手 2026-07-03 09:27:46

我用 `ar002...flows.csv` 跑一个初始版本，先不保存逐条预测，避免输出太大。这个结果可以直接告诉我们：top-K 是否真的比 all features 更好，以及哪些 flow 属性排在前面。

助手 2026-07-03 09:32:08

这个 Codex bundled Python 里没有 `scikit-learn`，所以本地轻跑卡在依赖上了；你的项目 `requirements.txt` 里本来就有 `scikit-learn`，Colab/你的训练环境应该能跑。为了不让脚本报一串难懂错误，我会再补一个依赖检查提示；同时给你准备好服务器/Colab 上直接运行的命令。

助手 2026-07-03 09:32:31

我发现你机器上还有一个本地 Python，不是 Microsoft Store 的占位符。我试一下这个环境有没有 sklearn；如果有，就可以直接把初始结果跑出来。

助手 2026-07-03 09:34:09

本地 Python 有 sklearn 1.8，可以跑。我现在用它执行脚本，先只跑 `logreg` 和 `extratrees`，这能快速判断“部分特征是否改善 all features”。

助手 2026-07-03 09:38:28

完整 top-K + ExtraTrees 跑得比预期久，4 分钟超时了。它可能已经写了一部分 ranking/中间文件；我先检查输出目录。如果 ranking 已经出来，我们至少能看到哪些属性有用；然后我用更轻的参数再跑一版 baseline。

助手 2026-07-03 09:38:44

超时后那个脚本进程还在跑，我先只停止刚刚启动的那一个进程，避免它继续占 CPU；其它你之前开的 Python 进程我不碰。

助手 2026-07-03 09:38:56

ranking 文件已经生成了，所以最重要的“哪些属性有用”已经有答案。我现在读前 30 个特征，并把脚本性能瓶颈改一下：减少默认树数量、允许只跑 ranking 或只跑轻量 baseline。

助手 2026-07-03 09:39:10

从已生成的 ranking 看，最强的不是“所有统计平均用上”，而是非常集中在 SYN、初始窗口、反向包头/长度、反向 IAT 这些属性上。这已经支持你的怀疑：只有部分 flow feature 真的有用。现在我用更轻的 logreg 版跑完 top-K vs all 的可比结果。

助手 2026-07-03 09:43:44

又超时了，说明耗时主要不是模型本身，很可能是互信息/相关性选择对全量训练集比较慢。我会停掉这个进程，然后把脚本优化成可以复用已经生成的 ranking，并增加 `--ranking\_method existing` 或 `--ranking\_only` 这类模式，避免每次都重新算。

助手 2026-07-03 09:43:59

这个慢点明确了：在构建 top-K 时逐个算 Spearman 相关太慢。最简单的做法是这次先用 `--corr\_threshold 1.0` 关闭去相关选择，直接按 ranking 取 top-K；之后如果需要再单独做去相关。

助手 2026-07-03 09:44:12

我会补两个小改动：默认不做去相关（避免慢），并允许用 `--ranking\_csv` 复用已经生成的 ranking。这样你可以先快速完成 baseline，再决定是否更需要更精细的去相关 top-K。

助手 2026-07-03 09:44:33

改好了。现在复用刚才已经算出的 ranking，再快速跑 top-K logreg baseline。

助手 2026-07-03 09:45:46

语法检查通过了，也没有我刚才启动的残留进程了。现在跑复用 ranking 的版本，应该只剩训练和评估。

助手 2026-07-03 09:51:13

复用 ranking 仍然超过 4 分钟，说明 `logreg` 在这个不平衡数据上收敛比较慢，不是 ranking 的问题。接下来我不再用 logreg 做本地 smoke test，改用更快的树模型并把树数降下来；正式实验你可以用 XGB/LGBM 跑。

助手 2026-07-03 09:51:25

它已经完成了几个 top-K，但结果表目前是“全部结束后才保存”，这不太友好。我先停掉这次，再把脚本改成每跑完一个 feature set 就增量保存结果，这样长实验中途也不会白跑。

助手 2026-07-03 10:14:04

它刚好自己结束了。现在我检查是否已经写出结果表；如果没有，我仍然会做增量保存的小修。

助手 2026-07-03 10:14:15

结果表已经正常产出了。接下来我读一下 top-K baseline 的结果，看“部分特征”有没有比 all features 改善。

助手 2026-07-03 10:14:28

本地环境居然也装了 XGBoost 和 LightGBM。为了更贴近你原来的 baseline，我再跑一版小规模但有意义的：`top40/top60/all`，模型用 XGB/LGBM，树数稍微降一点做快速诊断。这个结果会更接近你关心的“重跑 baseline 是否改善”。

助手 2026-07-03 10:23:36

可以，而且从我刚跑的结果看：**确实只有部分 flow feature 更有用**。

我新增了脚本：

```
[flow_feature_selection_baseline.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/flow_feature_selection_baseline.py:1)
```

它直接读原始 flow CSV，所以会保留属性名，然后做：

1. CSV flow feature
2. feature ranking
3. feature_ranking.csv
4. top-K feature baseline
5. all features

我已经用这个文件跑了：

```
C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/dataset/ar002_et12_20260511_002-stage1_flows.csv
```

****最有用的前几项 feature 是：****

1. syn_flag_count
2. init_win_bytes_forward
3. bwd_header_length
4. fwd_header_length
5. bwd_packet_length_max
6. init_win_bytes_backward
7. bwd_packet_length_std
8. avg_bwd_segment_size
9. total_backward_packets
10. total_length_of_bwd_packets

这说明有用信息主要集中在：

```
TCP SYN/flag  
initial window
```

```
IAT / active time
```

不是所有 flow 统计都同等有用。

我也重跑了一版 XGB/LGBM 快速 baseline：

```
top60 lgbm | P1=0.6169 | R1=0.7330 | F1=0.6699 | PR_AUC=0.7473  
all lgbm | P1=0.6430 | R1=0.6832 | F1=0.6625 | PR_AUC=0.7386  
  
top60 xgb | P1=0.5964 | R1=0.7294 | F1=0.6562 | PR_AUC=0.7045  
all xgb | P1=0.5746 | R1=0.7550 | F1=0.6525 | PR_AUC=0.7026
```

结论: **top60 比 all features 略好, 尤其 LGBM 的 F1 从 0.6625 提到 0.6699, PR_AUC 从 0.7386 提到 0.7473。** 提升不大, 但方向是对的: 删掉一部分弱/噪声 feature 有帮助。

结果文件在这里:

```
C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/diagnostics/ar002_flow_feature_selection_xgb_lgbm_fast/
```

主要看这两个:

```
feature_ranking.csv
feature_selection_baseline_results.csv
```

你之后可以这样正式跑:

```
python s1/flow_feature_selection_baseline.py `
--flow_csv dataset/ar002_et12_20260511_002-stage1_flows.csv `
--out_dir s1/diagnostics/ar002_flow_feature_selection_full `
--models xgb lgbm `
--top_k 10 20 30 40 50 60 all
```

如果想加入 `protocol` 或端口特征, 可以加:

```
--include_protocol --include_ports
```

但我建议论文主实验默认不要用 IP, 端口也要谨慎, 因为容易学到环境偏差。

用户 2026-07-03 11:34:41

如果使用方案A, 有可能性能会好过现在最强的方案B吗?

助手 2026-07-03 11:34:58

可以, 有可能, 但我会把它判断为“可能小幅超过 B, 不应该期待稳定大幅超过 B”。原因很朴素: A 的确给每个 packet token 加了全局 flow context, 但你的 flow-only 结果说明这些 flow feature 单独判别力弱, 而且很多 top feature 其实已经能从 packet sequence 里推出来。

助手 2026-07-03 11:35:20

有可能, 但**概率不是特别高**。更准确地说:

```
A      >      B
      /      /
class1 F1=0.83
```

你现在最强 B 大概是:

```
B seq128: P1=0.7881, R1=0.7825, F1=0.7853
```

而 flow-only baseline 大概只有:

```
XGB/LGBM: F1≈0.66-0.67
```

这说明 flow feature 有信号, 但比 packet sequence 弱很多。所以方案 A 想超过 B, 必须满足一个条件:

```
flow feature      B      FN/FP      B
```

为什么方案 A 可能提升

方案 A 是:

```
packet token = [packet_feature ; flow_feature]
```

它的优点是模型可以学习这种关系:

packet pattern flow context

比如：

```
tcp_syn      syn_flag_count      flow
            bwd_iat/      backward response      flow
            tcp_window      flow duration / packet count
```

所以 A 理论上比简单 flow-only 更强，因为它不是单独用 flow，而是让 flow feature 调节 packet token 的含义。

****为什么方案 A 也可能不如 B****

你的 top feature ranking 里最强的是：

```
syn_flag_count
init_win_bytes_forward
bwd_header_length
fwd_header_length
bwd_packet_length_max
init_win_bytes_backward
bwd_packet_length_std
total_backward_packets
```

其中很多信息 B 已经部分看到了：

```
packet      tcp_syn
packet      tcp_window
packet      tcp_header_len
packet      pkt_len/ip_len/payload_len
packet      direction
packet      flow_iat_us
```

所以 flow feature 对 B 来说可能不是全新信息，而是“聚合版重复信息”。如果重复信息很多，A 的提升就会很有限。

另外 A 把同一个 flow feature 复制到每个 packet token 上，风险是：

```
flow feature                      flow branch
packet-level temporal pattern
                    flow statistics
```

所以 A 不是一定比 C 好。A 的 interaction 更强，但风险也更高；C residual-gated 更稳。

****我的判断****

如果你直接用所有 flow feature 跑 A：

```
B
F1 0.77-0.79
```

如果你只用筛选后的 top flow feature 跑 A：

```
B
F1 0.785-0.80
```

如果你想靠 A 到 0.83：

```
overlap analysis      flow feature      B      FN/FP
```

****最值得跑的 A 版本****

我建议不要跑“全量 flow feature A”，而是跑这几个：

```
A-top20
A-top40
A-top60
A-top60 + seq128
A-top60 + seq256
```

优先使用你刚筛出来的 top features。尤其是：

```
syn_flag_count
init_win_bytes_forward
bwd_header_length
fwd_header_length
bwd_packet_length_max
init_win_bytes_backward
bwd_packet_length_std
avg_bwd_segment_size
total_backward_packets
total_length_of_bwd_packets
packet_length_std
max_packet_length
bwd_iat_total
bwd_iat_max
active_max
bwd_iat_mean
```

****实验上怎么判断 A 有没有价值****

不要只看最终 F1。你要看这三个东西：

```
1. A      B      FN
2. A      B      FP
3. A              B      A
```

如果 A 的结果是：

```
FN      FP
```

那 A 有价值。

如果是：

```
FN      FP
```

那 A 不适合。

如果是：

```
P      R
```

说明 flow feature 更像 false-positive filter，不是 recall booster。

****最终建议****

你可以跑 A，但我不建议把 A 当作主救命路线。更合理的顺序是：

```
1. B      baseline
2.  A-top40 / A-top60      fusion ablation
3.  C residual-gated-top60
4.  B + XGB/LGBM stacking
```

如果 A 超过 B，就写：

```
Early fusion of selected flow statistics provides complementary global context.
```

如果 A 没超过 B，也完全合理，可以写：

```
Naive early fusion of flow statistics does not consistently improve packet-level temporal modeling,
suggesting that packet sequence features already capture much of the discriminative flow-level information.
```

所以答案是：****有可能超过 B，但不要直接用 all flow features；用筛选后的 top flow features 跑 A，才是最有希望的版本。****

用户 2026-07-03 12:00:25

仔细阅读s2下面的代码，是我stage2的代码，将s1的输出作为输入，然后使用不同的flow context (time only, same source, same destination, endpoint) 做transformer分类。

仔细阅读我的代码？检查是否存在问题

助手 2026-07-03 12:00:36

我来按 code review 的方式读`s2`：先梳理入口、dataset/context

构造、split、model/trainer/eval，再重点找数据泄漏、context 泄漏、shape/mask、threshold 和指标问题。

助手 2026-07-03 12:00:56

`rg` 输出里混了很多 notebook 和历史 artifacts。我现在聚焦主路径：`run\_stage2.py` 加

`s2/stage2/\*.py`，再对当前 config 看具体参数。重点是检查 context 构造是不是只用过去 flow、split 是否泄漏、target flow 是否被放进自己的 context。

助手 2026-07-03 12:01:09

核心 context 构造看起来是“先取历史，再 append 当前

flow”，这一点方向是对的。但我已经看到一个潜在大问题：`train\_only\_for\_eval` 现在对 train split 也只允许 train 历史，且如果 target 本身是 val/test，`include\_target=True` 会把 val/test 当前 token

加进去；这要结合模型目标判断是否合理。我继续看 dataset/model/trainer。

助手 2026-07-03 12:01:27

我已经看到几处需要确认的点：训练中使用固定`threshold=0.5`来选 best epoch，最后才在 val 上自动选阈值；这会导致“选模型”和“最终指标”优化目标不一致。还有 AMP/GradScaler 在 CPU 上的兼容性需要检查。继续读 metrics/loss/config。

助手 2026-07-03 12:01:46

`Stage2Dataset` 的索引对齐基本合理：`meta\_df` 排序后，`z\_sorted` 同步重排，context index 使用排序后的行号。模型里 last pooling 对 left padding 也处理了。现在我看 config，因为很多问题会来自默认参数和实际运行参数。

助手 2026-07-03 12:02:03

配置里`time\_only`用`online`，而 source/destination/endpoint 用

`split\_isolated`。这两者回答的是不同实验问题：`online` 更接近真实流式检测；`split\_isolated` 更像每个 split 单独构图，避免跨 split 上下文，但会让 test 第一个 flow 没有 train 历史。这个需要在报告中明确，否则不同 context 不公平。

助手 2026-07-03 12:02:14

我准备做一次`py\_compile`检查，先排除语法层面的硬错误；然后我会用静态阅读结果整理成 findings。注意我不会改代码，只做检查。

助手 2026-07-03 12:02:36

语法检查通过了。现在我再快速看已有 Stage2 artifacts 的配置和指标，看看代码问题有没有在实际结果里留下痕迹，比如 test 用了异常阈值、context 长度分布异常、train/val/test 指标反常。

助手 2026-07-03 12:02:58

现有 artifact 只有两组 time-only split 结果。我会直接读 summary，比 JSON 更快看出整体表现。

助手 2026-07-03 12:04:08

我已经有足够多的 findings 了。最后再扫一下所有 YAML 里的 `metric\_for\_best` 和 `threshold\_metric`，避免漏掉会直接报错的配置。

助手 2026-07-03 12:04:13

PowerShell 里 `rg` 直接吃 Windows 通配路径有点别扭，我换成 `Get-ChildItem` 管道扫 YAML。

助手 2026-07-03 12:05:04

我仔细读了 `s2` 主路径代码：`run\_stage2.py`、`stage2/context.py`、`data\_io.py`、`dataset.py`、`model.py`、`trainer.py`、`metrics.py`、`losses.py` 和主要 YAML。语法检查通过，但有几个会影响实验可信度的问题。

****主要问题****

1. ****CLI 默认值会覆盖 YAML 配置****

在 [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:59) 里：

```
--seed default=42
--model_pooling default="last"
--cls_head default=1
```

然后 [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:133) 会无条件覆盖 config。

这意味着即使你的 YAML 写了：

```
seed: 130
cls_head: 3
pooling: attention
```

只要命令行没显式传参，也会被改成：

```
seed: 42
cls_head: 1
pooling: last
```

这是高优先级 bug。应该把这些 argparse 默认值改成 `None`。

2. ****`online` context 可能产生 split 泄漏****

[context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:43) 中：

```
if context_policy == "online":
    return True
```

而 [data_io.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/data_io.py:178) 会把 train/val/test 合并后按时间重新排序。

如果 Stage1 split 不是严格 chronological, 那么 train 样本的历史 context 里可能包含更早出现的 val/test flow embedding。这不是 label 泄漏, 但属于 evaluation data embedding 泄漏。只有当 Stage1 split 本身是严格时间切分时, `online` 才比较安全。

3. ****Transformer 位置编码 + 动态 left padding 会让输出依赖 batch 组成****

[dataset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/dataset.py:64) 每个 batch 动态取 `max\_len`, 并 left padding。然后
[model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:561) 在 padding 后直接加位置编码。

结果是: 同一个样本在不同 batch 里, 真实 token 的 position index 可能不同, 模型输出会依赖 batch 里最长序列是谁。你的部分 config 已经设置 `use\_positional\_encoding: false`, 这是对的。若要开位置编码, 建议固定 pad 到 `window\_size + 1`, 或者先 compact 有效 token 再加位置编码。

4. ****`metric\_for\_best: val\_f1` 在当前 metrics 里不成立****

当前 [metrics.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/metrics.py:128) 输出的是:

```
macro_f1
weighted_f1
f1_label1
```

没有 `f1`。但默认配置

[config.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/config.py:63) 是:

```
metric_for_best: val_f1
```

`metric\_value()` 找不到这个 key 时会返回 `-inf`, 可能导致 best checkpoint 不正常。你现在部分 YAML 用 `f1\_label1`, 这个是对的。建议统一用:

```
metric_for_best: val_f1_label1
```

或:

```
metric_for_best: val_macro_f1
```

5. ****`threshold\_metric: f1` 会报错****

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:269) 只支持:

```
f1_label1
recall
precision
macro_f1
```

但 [stage2_no_context.yaml](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2_no_context.yaml:51) 和 [stage2_lstm_context.yaml](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2_lstm_context.yaml:46) 用的是:

```
threshold_metric: f1
```

这会在 final evaluation 阶段直接 `ValueError`。应该改成:

```
threshold_metric: f1_label1
```

6. ****训练选 best epoch 用固定 0.5 阈值, 但最终测试用 val 自动阈值****

训练阶段 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:336) 用固定 threshold 评估 val, 最后

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:469) 才在 val 上找最佳 threshold。

这会导致：

```
best_checkpoint      threshold
```

尤其 class1 F1 对 threshold 很敏感。建议训练中监控 `val\_pr\_auc` 或每个 epoch 都做 val threshold search 后再选 `val\_f1\_label1`。

7. ****默认同时使用 weighted sampler 和 class-weighted focal loss****

[config.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/config.py:64) 默认：

```
use_weighted_sampler: true
class_weighted_loss: true
```

而 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:65) 注释里也写了不建议同时用。两者叠加会过度强调 class1，容易 recall 上去、precision 掉很多。建议二选一。

8. ****`train\_only\_for\_eval` 在大数据上可能拿不到 train context****

[context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:87) 先用有限 `history\_cap` 截断历史，再在

[context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:122) 按 split 过滤。

如果 test 很晚、train 很早，train flow 可能已经被 deque 挤掉，过滤后 context 为空。对于 `train\_only\_for\_eval`，应该先维护 train-only history，或者过滤逻辑前不要用太小的 cap。

****相对正确的部分****

你的 Stage2 主体思路是成立的：

```
Stage1 embedding z ->          ->          context -> Transformer/LSTM
```

而且 [context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:125) 是先构建 context，再把当前 flow 写入历史，避免了 self/future leakage。`include\_target=True` 把当前 flow embedding 放到最后也合理，因为目标分类必须看当前 flow 自身表示。

****我建议优先修这几个****

1. 把 `run\_stage2.py` 里 `seed/model\_pooling/cls\_head` 默认值改成 `None`。
2. 统一 YAML: `metric\_for\_best: val\_f1\_label1`, `threshold\_metric: f1\_label1`。
3. 如果用 Transformer，默认 `use\_positional\_encoding: false`，或者固定 padding 长度。
4. 保证所有 context 方法使用同一个 `context\_policy` 做公平对比。
5. 不要同时开 `use\_weighted\_sampler` 和 `class\_weighted\_loss`。
6. 训练阶段改成用 val threshold search 后的 F1 或 PR_AUC 选 best epoch。

整体判断：代码不是“不能用”，但现在有几处会让 Stage2 结果不稳定或不公平。最危险的是 CLI 覆盖 YAML、`online` 在非时间 split 下的泄漏风险、以及动态 padding + positional encoding。

用户 2026-07-03 13:13:41

Files mentioned by the user:

```
## Wednesday-workingHours.pcap_ISCX.csv: C:/Users/XiaoyanXiong/Downloads/MachineLearningCSV/MachineLearningCVE/Wednesday-workingHours.pcap_ISCX.csv
```

My request for Codex:

这是公共数据集的csv，我想你仔细分析这个文件，然后给我生成一个baseline的代码。只做二分类

助手 2026-07-03 13:13:55

我会先按 CSV/表格处理流程读这个公共数据集的列名、标签分布、缺失/Inf 情况，然后给你生成一个二分类 baseline 脚本。这个脚本会直接处理 CICIDS 这种带空格列名、`Infinity/NaN`、多攻击标签的 CSV。

助手 2026-07-03 13:15:23

我先把这个公共 CSV 的列名、标签分布、缺失/无穷值这些关键点摸清楚，然后把二分类 baseline 写成一个可复用脚本。这样后面你换别的 CICIDS CSV 也能直接跑。

助手 2026-07-03 13:15:39

这个文件本身是典型 CICIDS2017 的 ML CSV：列名前面有很多空格，而且 `Fwd Header Length` 在表头里重复出现一次。baseline 代码里我会专门处理这些小坑，否则很容易出现“看起来在跑，其实列名/重复列/Infinity 没清干净”的问题。

助手 2026-07-03 13:16:25

分析结果比较清楚：这个 Wednesday 文件有 692,703 条 flow，78 个候选特征，二分类里攻击占 36.48%。两个列有 `inf`，一个列有 NaN，另外有 10 个恒定无信息列；这些都会写进清洗流程里。

助手 2026-07-03 13:16:56

接下来我会在 `s1` 下面新增一个独立脚本，不改你现有 Stage1/Stage2 训练代码。它会做三件事：清洗 CICIDS CSV、按 BENIGN/Attack 做二分类、在验证集选 class1 F1 最优阈值后报告测试集结果。

助手 2026-07-03 13:20:03

我刚写完主体脚本，现在做一次代码级检查。顺手修一个细节：`Flow Bytes/s` 和 `Flow Packets/s` 里的 Infinity 需要在报告里真实计数，不能只在清洗时默默变成 NaN。

助手 2026-07-03 13:20:29

编译发现一个很 Windows 的问题：docstring 里的 `C:\Users\...` 被 Python 当成 Unicode 转义了。我把示例字符串改成 raw docstring，再继续跑。

助手 2026-07-03 13:20:58

为了验证脚本又不等完整模型训练，我加一个 `--sample\_rows` 参数：它会在读完整 CSV 后按二分类标签抽样。这样 smoke test 能同时包含 BENIGN 和 attack，不会因为文件前半段全是正常流量而训练失败。

助手 2026-07-03 13:22:16

smoke test 已经过了：抽样 20,000 行时脚本能完成清洗、切分、阈值搜索和结果保存。这个抽样结果不能当论文结果，只是证明代码路径是通的；公共 CICIDS 的随机切分通常会非常高，后面报告里要说明 split 策略。

助手 2026-07-03 13:22:24

我顺手把 LogisticRegression 的一个新版 sklearn warning 去掉了，避免你正式跑的时候日志里出现无关噪音。

助手 2026-07-03 13:22:49

我会再跑一个完整数据集的 LightGBM baseline，作为“这份 CSV 在标准随机二分类切分下大概能到什么水平”的参考。完整多模型脚本已经写好了；这一轮先跑一个强而快的模型，避免让你等太久。

助手 2026-07-03 13:30:20

完整 LightGBM 已经跑完了：随机 stratified 切分下几乎满分，这和 CICIDS 公共 ML CSV 的常见现象一致。这个结果适合作为“公共数据集 sanity-check baseline”，但不能直接拿来证明模型在真实时间泛化上也这么强。

助手 2026-07-03 13:30:59

我还看了一下保存的报告：原始表头有 79 列，但因为 `Fwd Header Length` 重复，脚本默认丢掉 `.1` 那一列，所以进入建模前是 78 列，其中 77 个数值候选特征，训练集再去掉 10 个恒定特征，最终 67 个特征。

助手 2026-07-03 13:31:10

我把报告字段也补全了：之后新跑出来的 `dataset\_report.json` 会同时记录 raw column count 和去重后的 column count，避免你写报告时数字对不上。

助手 2026-07-03 13:31:28

工作区本来就有很多未提交/新增文件，我只新增了这个 public CSV baseline 脚本，并让脚本生成了两个结果目录。现在再做一个小的类型标注清理，保证脚本后续维护起来舒服一点。

助手 2026-07-03 13:32:22

已完成。新增脚本在这里：

```
[s1/cicids_binary_baseline.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/cicids_binary_baseline.py:1)
```

我对这个公共 CSV 的关键分析结果：

- 原始文件：692,703 行，79 列，约 214.74 MB
- `Label` 分布：
- `BENIGN`：440,031
- `DoS Hulk`：231,073
- `DoS GoldenEye`：10,293
- `DoS slowloris`：5,796
- `DoS Slowhttptest`：5,499
- `Heartbleed`：11
- 二分类后：class0=440,031，class1=252,672，攻击比例约 36.48%
- 数据问题：
- `Fwd Header Length` 表头重复，脚本默认丢掉 `.1`
- `Flow Packets/s` 有 1,297 个 `inf`
- `Flow Bytes/s` 有 289 个 `inf`，清洗后总缺失/异常为 1,297
- 10 个恒定特征会被自动删除

我也跑了完整 LightGBM baseline, 结果保存在:

```
`C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\cicids_wednesday_binary_lgbm_full`
```

完整随机 stratified split 下结果:

```
lgbm | P1=0.9996 | R1=0.9999 | F1_1=0.9998 | PR_AUC=1.0000 | AUC=1.0000 | threshold=0.4340  
confusion_matrix = [[87988, 18], [7, 50528]]
```

运行完整多模型 baseline:

```
C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\cicids_binary_baseline.py --csv_path C:\Users\XiaoyanXiong\Downloads\MachineLearningCSV\MachineLearningCVE\Wednesday-workingHours.pcap_ISCX.csv --out_dir C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\cicids_wednesday_binary_all --models logreg extratrees xgb lgbm --save_predictions
```

如果你想避免 `Destination Port` 带来的端口泄漏/过拟合, 可以加:

```
--no-include_ports
```

注意: 这个公共 CICIDS ML CSV 在随机切分下几乎满分是常见现象, 说明这个公共数据集 flow feature 很容易被传统模型分开。它适合作为 sanity-check baseline, 但不能直接证明真实部署或跨时间泛化也这么强。

用户 2026-07-03 13:56:34

项目中dataset/Wednesday-workingHours-stage1_flows.csv这是我用自定义的suricata插件从CICIDS PCAP中提取出的features。请给我生成一个baseline的代码。只做二分类

助手 2026-07-03 13:56:46

我会用表格/CSV 处理那套流程来做: 先看你这个 Suricata flow CSV 的真实列名和标签格式, 再生成一个专门适配自定义插件输出的二分类 baseline 脚本。

助手 2026-07-03 13:56:57

这个任务不需要生成 Excel 工作簿, 所以我不会走 artifact workbook 流程; 这里直接用 pandas/ sklearn 写可复现的训练脚本更合适。先读取你的自定义 flow CSV 的头部和标签分布。

助手 2026-07-03 13:57:09

这个文件和公共 ML CSV 不一样: 它有 `flow\_id`、时间戳、IP、端口、协议, 以及你插件输出的 numeric label。baseline 里我会默认丢掉 `flow\_id/source\_ip/destination\_ip/timestamp`, 保留端口和协议可配置。

助手 2026-07-03 13:57:49

我已经读到这份自定义 flow CSV: 475,809 条 flow, 77 列, label 是数字 0/1, class1 占 44.17%。没有 NaN/Inf, 只有 3 个恒定特征; `flow\_id` 有 242 个重复, baseline 会默认把它当标识符丢掉, 不作为模型输入。

助手 2026-07-03 13:58:09

现在我会新增一个独立脚本, 不碰你已有训练代码。这个脚本的默认输入就是 `dataset/Wednesday-workingHours-stage1\_flows.csv`, 输出会放到 `s1/results/...`, 并保存 dataset report、metrics、feature importance 和可选 prediction CSV。

助手 2026-07-03 14:00:59

脚本已经生成了。我现在做编译检查和一次快速训练验证，先用少量树跑 `logreg + extratrees`，确认清洗、切分、阈值搜索、保存输出都正常。

助手 2026-07-03 14:04:38

快速验证已经跑通，而且这份自定义 Suricata flow feature 在随机切分下也很强：50 棵 ExtraTrees 的 class1 F1 约 0.9968。这个是 smoke/快速参数，不是最终建议写进论文的完整多模型结果。

助手 2026-07-03 14:04:56

我再跑一个完整 LightGBM 版本作为可直接参考的 baseline；它通常比线性模型更适合这类 flow 统计特征。

助手 2026-07-03 14:10:01

完整 LightGBM 也跑完了，结果非常高：class1 F1 约 0.9980。接下来我看一下保存的 report 和重要特征，确保最后给你的路径和命令是准确的。

助手 2026-07-03 14:10:23

注意一个值得写进分析的点：完整 LGBM 的最高重要特征是 `source\_port`，这可能带有数据集端口模式信息。为了避免 baseline 过于“占便宜”，我再跑一个 `--no-include\_ports` 版本，看看去掉端口后性能是否仍然稳。

助手 2026-07-03 14:15:12

去掉端口后 LightGBM 仍然几乎不掉：class1 F1 约 0.9979。这说明你的自定义 Suricata flow features 本身已经足够强，不是完全靠端口字段撑起来的。

助手 2026-07-03 14:15:58

已生成并验证好了：

[s1/suricata_flow_binary_baseline.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/suricata_flow_binary_baseline.py:1)

这个脚本专门适配你的自定义 Suricata flow CSV，只做二分类：`label=0` 为 normal，非 0 为 attack。默认会丢掉 `flow\_id`、时间戳、`source\_ip/destination\_ip`，保留端口和协议；会自动去掉恒定特征，做 median impute、RobustScaler、验证集阈值搜索，然后在 test 上输出 class1 的 precision/recall/F1、AUC、PR-AUC、confusion matrix。

我也已经跑了完整 LGBM baseline：

```
with ports:
P1=0.9986 | R1=0.9973 | F1_1=0.9980 | PR_AUC=1.0000 | AUC=1.0000
CM = [[53068, 57], [115, 41922]]

no ports:
P1=0.9990 | R1=0.9968 | F1_1=0.9979 | PR_AUC=1.0000 | AUC=1.0000
CM = [[53081, 44], [133, 41904]]
```

结果目录：

`C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\wednesday\_suricata\_flow\_binary\_lgbm\_full`

```
`C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\wednesday_suricata_flow_binary_lgbm_full_no_ports`
```

推荐正式多模型运行命令：

```
C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\suricata_flow_binary_baseline.py --flow_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\Wednesday-workingHours-stage1_flows.csv --out_dir C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\wednesday_suricata_flow_binary_all --models logreg extratrees xgb lgbm --save_predictions
```

更公平的无端口版本：

```
... --models lgbm --no-include_ports
```

注意：这是 random stratified split，结果会比较乐观。写报告时可以把它作为 flow-feature sanity-check baseline；如果要强调真实泛化，建议再跑 `--split chronological`。

用户 2026-07-03 14:19:15

这个PCAP，baseline结果很好。比我的好太多了

助手 2026-07-03 14:19:39

对，这个现象其实很重要：**不是你的 Suricata feature 抽错了，而是这个 CICIDS Wednesday PCAP 本身太容易被 flow-level baseline 分开了。**

你现在看到的是：

```
Suricata    flow feature + LGBM
class1 F1 ≈ 0.9980

    source_port / destination_port
class1 F1 ≈ 0.9979
```

这说明两件事：

1. 你的自定义 Suricata 插件在这个 PCAP 上抽出来的 flow features 是有效的。
2. 这个 PCAP 的二分类任务太“干净”：DoS 类攻击和 BENIGN 在 flow duration、IAT、packet length、window size、backward length 等统计特征上差异非常大。

所以它比你之前的数据好很多，不代表你的 Transformer 差，也不代表你之前的数据一定错。更可能是：

- CICIDS Wednesday 是经典 benchmark，攻击模式集中，二分类很容易。
- 你的原始实验数据更复杂，class1 可能更隐蔽、更混杂、标签噪声更大。
- random split 会把同一攻击阶段/同一流量模式同时分到 train/val/test，baseline 会学得非常轻松。
- flow-level feature 对 DoS 很强，但对你原来那种更真实/更细粒度的异常可能没那么强。

下一步我建议你一定跑这三个对照：

```
# 1.
C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\suricata_flow_binary_baseline.py --flow_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\Wednesday-workingHours-stage1_flows.csv --out_dir C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\wednesday_suricata_flow_binary_chrono --models lgbm --split chronological

# 2.
... --models lgbm --no-include_ports

# 3.
... --models lgbm --no-include_ports --no-include_protocol
```

论文里可以这样写逻辑：

****Public CICIDS sanity check:**** 自定义 Suricata 插件提取的 flow features 在公开 CICIDS Wednesday PCAP 上可以复现传统 ML 的高性能，说明 feature extraction pipeline 是有效的。

****Main research dataset:**** 在更困难的数据上，flow-only baseline 明显下降，因此需要 packet-sequence Transformer 和 stage2 context modeling。

这反而是一个好结果：它帮你证明“插件没问题”，同时也解释了为什么你的主任务需要更复杂模型。

用户 2026-07-03 14:23:27

dataset/Wednesday-workingHours-stage1_flows.csv你在写这个的baseline的时候，有没有像我stage1的数据处理一样，log1处理所有numerical？

助手 2026-07-03 14:23:44

我先确认一下你 Stage1 里 flow feature 的预处理到底怎么做，然后对照我刚写的 baseline 脚本回答你。这个点很关键：如果 baseline 和 Stage1 预处理不一致，结果解释会有偏差。

助手 2026-07-03 14:24:01

我看到一个细节：Stage1 代码本身不是“无条件对所有 numerical 做 log1p”，而是根据 YAML 里的 `preprocessing.numerical.log1p.flow` 和 `packet` 列表做 selected log1p。现在我再看你当前配置里这个列表到底是不是覆盖了全部 flow numerical。

助手 2026-07-03 14:24:17

结论先说：我刚写的 `suricata\_flow\_binary\_baseline.py` 之前没有做 Stage1 那种 log1p，它做的是 numeric conversion → median impute → RobustScaler → clip。这个确实和你的 Stage1 flow 预处理不完全一致，我现在把脚本补上可配置的 log1p 模式。

助手 2026-07-03 14:24:38

我会按你 Stage1 的真实逻辑来补：不是对端口、protocol、flag 这类字段乱做 log1p，而是对 YAML 里那些 duration/count/length/rate/IAT/window/active/idle 类 flow 数值做 selected log1p。也会留一个 `--log1p\_mode all\_nonnegative` 给你做额外对照。

助手 2026-07-03 14:25:23

已补上 `--log1p\_mode`，默认现在是 `stage1\_selected`，也就是对齐你 Stage1 YAML 里的 flow log1p 列表。接下来我编译并跑一个小验证，确认脚本没被这个改动弄坏。

助手 2026-07-03 14:26:59

这个 smoke test 显示现在确实用了 52 个 Stage1-style log1p flow 列。为了不让之前的 LGBM 数字和新脚本默认设置混在一起，我再跑一个完整 LGBM 的 `stage1\_selected log1p` 版本，给你一个可引用的新结果。

助手 2026-07-03 14:31:29

完整 LGBM 也跑完了：加入 Stage1-style log1p 后 F1 仍然约 0.9981，和之前几乎一样。说明这个 PCAP 的 flow-level 可分性非常强，预处理差异不是高性能的主要原因。

助手 2026-07-03 14:31:50

你说得对，**我一开始写的 baseline 没有像 Stage1 那样做 log1p**。之前版本是：

```
numeric conversion -> median impute -> RobustScaler -> clip
```

而你的 Stage1 真实逻辑是：

```
numeric conversion -> replace inf -> selected log1p -> fillna -> quantile clipping -> scaler
```

而且 Stage1 不是对所有 numerical 都 log1p，是对 YAML 里指定的 heavy-tailed flow columns 做 log1p，比如 duration、packet count、length、IAT、rate、window、active/idle 等；不会对 `source\_port`、`destination\_port`、`protocol`、flag/ratio 这些乱做 log1p。

我已经把脚本修好了：

```
[s1/suricata_flow_binary_baseline.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/suricata_flow_binary_baseline.py:79)
```

现在默认：

```
--log1p_mode stage1_selected
```

也就是对齐 Stage1 YAML 的 flow log1p 列表。这次在 `Wednesday-workingHours-stage1\_flows.csv` 上实际用了 52 个 log1p columns。

我重新跑了完整 LGBM：

```
log1p_mode=stage1_selected
P1=0.9980 | R1=0.9981 | F1_1=0.9981 | PR_AUC=1.0000 | AUC=1.0000
CM = [[53043, 82], [79, 41958]]
threshold = 0.226
```

结果目录：

```
`C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\wednesday_suricata_flow_binary_lgbm_full_stage1_log1p`
```

所以结论是：**之前没做 log1p，但补上后结果几乎不变**。这说明这个 PCAP 的 flow-level pattern 本身非常容易分，不是因为没/有 log1p 才高。

现在你可以这样明确跑：

```
C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\
suricata_flow_binary_baseline.py --flow_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\Wednesday-
workingHours-stage1_flows.csv --out_dir C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\results\
wednesday_suricata_flow_binary_all_stage1_log1p --models logreg extratrees xgb lgbm --log1p_mode
stage1_selected --save_predictions
```

小提醒：现在 log1p 已对齐 Stage1，但 baseline 还不是 100% 复用 `Stage1Preprocessor`，因为它仍然用 median impute 和固定 clip。要做“完全同 Stage1 preprocessing”的 baseline，下一步应直接复用 `stage1/preprocessing.py` + YAML 配置。

用户 2026-07-03 15:32:23

我的stage1中有 has_external_test = external_packet_csv is not None and external_flow_csv is not None 在外部数据集上进行测试。但是我现在dataset/Wednesday-workingHours-stage1_flows.csv这个数据集太大了，我只想截取一部分5000条左右，里面一定要有malicious，我该如何截取？

请结合我现有代码，给我提供代码

助手 2026-07-03 15:32:42

我会把这个做成一个小脚本：先从 flow CSV 中按 `flow\_id` 抽 5000 个 flow，确保包含/控制 malicious 数量，然后从超大的 packet CSV 里用 chunk 只筛出这些 `flow\_id` 对应的 packets。这样就能直接作为 Stage1 的 `external\_packet\_csv/external\_flow\_csv` 使用。

助手 2026-07-03 15:32:58

Stage1 外部测试不是只需要 flow 子集，还需要匹配的 packet 子集。你的 pipeline 会先读 `external\_packet\_csv` 和 `external\_flow\_csv`，然后用 external flows 的 `flow\_id` 作为 test ids，所以两个 CSV 必须按同一批 `flow\_id` 截取。

助手 2026-07-03 15:34:15

我会额外处理一个你代码里的隐性坑：`generate\_and\_save\_stage1\_tensors()` 会用 `flow\_order.merge(flows, on=flow\_id)`，如果 external flow CSV 里有重复 `flow\_id`，可能导致 `labels` 和 `x` 数量不一致。所以截取脚本会默认先去重 `flow\_id`。

助手 2026-07-03 15:35:40

脚本已经写好。现在我做编译检查，然后实际跑一次默认 5000-flow 截取，输出到 `dataset/subsets`，这样你可以直接拿生成的两个 CSV 接 Stage1 external test。

助手 2026-07-03 15:37:52

生成成功了：5000 个 flow，其中 malicious/class1 有 2207 个，packet 子集只有约 6.6 万行，不再是 2.7GB 那种量级。我再做一次覆盖检查，确认 flow 和 packet 的 `flow\_id` 对齐。

助手 2026-07-03 15:38:21

可以，已经给你写好并实际生成了一份 5000-flow 子集。

代码在这里：

```
[s1/make_stage1_external_subset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/make_stage1_external_subset.py:1)
```

生成结果：

```
flow_csv:
C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\subsets\Wednesday-workingHours-external5000-stage1_flows.csv

packet_csv:
C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\subsets\Wednesday-workingHours-external5000-stage1_packets.csv
```

这次截取结果：

```
flows: 5000
normal: 2793
malicious: 2207
```

```
packets: 79832
duplicate_flow_id: 0
flow_ids_missing_packets: 0
packet_ids_missing_flows: 0
```

也就是说它可以直接作为 Stage1 external test:

```
C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\
run_stage1.py --packet_csv YOUR_TRAIN_PACKETS.csv --flow_csv YOUR_TRAIN_FLOWS.csv --config YOUR_CONFIG.yaml --
external_packet_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\subsets\Wednesday-workingHours-
external5000-stage1_packets.csv --external_flow_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\
subsets\Wednesday-workingHours-external5000-stage1_flows.csv --out_dir C:\Users\XiaoyanXiong\Desktop\
NIDSTransformer\s1\results\external_wednesday_5000
```

以后你想重新截取可以跑:

```
C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\
make_stage1_external_subset.py --flow_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\Wednesday-
workingHours-stage1_flows.csv --packet_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\Wednesday-
workingHours-stage1_packets.csv --out_dir C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\subsets --
prefix Wednesday-workingHours-external5000 --num_flows 5000
```

如果你想强制 malicious 占 50%:

```
--malicious_ratio 0.5
```

这个脚本专门适配你现有 Stage1 external-test 逻辑: 先抽 flow, 再按同一批 `flow\_id` 从大 packet CSV 里流式筛 packets, 并默认去掉重复 `flow\_id`, 避免你 Stage1 里 `flow\_order.merge()` 被重复 flow 撑坏。

用户 2026-07-03 16:06:05

我的stage1中有 has_external_test = external_packet_csv is not None and external_flow_csv is not None 在外部数据集上进行测试。但是我现在dataset/Wednesday-workingHours-stage1_flows.csv这个数据集太大了, 我只想截取一部分5000条左右, 里面一定要有malicious, 我该如何截取? stage1是明显的按时间split。所以我希望截取的也是按时间的。

请结合我现有代码, 给我提供代码

助手 2026-07-03 16:06:25

我先看一下 Stage1 外部测试入口是怎么读 `external\_packet\_csv/external\_flow\_csv` 的, 然后给你一个脚本: 从 flow CSV 按时间排序截取一个连续窗口, 并同步过滤 packet CSV, 保证可以直接作为 external test 输入。

助手 2026-07-03 16:06:38

你的 Stage1 外部测试确实需要两个文件: `external\_flow\_csv` 和 `external\_packet\_csv`。所以只截 flow 不够, 我会写成 “一次选 flow 窗口 + chunk 过滤 packet” 的脚本, 避免把 2.7GB packet CSV 整个读进内存。

助手 2026-07-03 16:07:35

我会让脚本默认去重 `flow\_id`, 因为这个 Wednesday flow CSV 里有 242 个重复 flow_id; 如果外部 test flow 文件里还保留重复 id, 你的 Stage1 后面用 flow_id merge 时有机会出奇怪问题。脚本会先按时间排序后取第一个重复项, 保证 external flow_id 唯一。

助手 2026-07-03 16:09:38

脚本已经写好。现在我先做编译检查, 再跑一个 `--skip\_packet\_filter` 的 dry run, 只生成 5000 条 flow 窗口和报告, 确认窗口里确实有 malicious。Packet 过滤要扫 2.7GB 文件, 稍后你需要时直接跑完整命令。

助手 2026-07-03 16:10:14

dry run 选出来的 5000 条 flow 是连续时间窗口，而且 label 分布很健康：normal 2793、malicious 2207。这个比“只找攻击最多的窗口”更适合作 external test，因为两类都有。

助手 2026-07-03 16:10:28

dry run 没问题。既然你是要给 Stage1 external test 用，我现在直接跑完整版本，生成匹配的 packet 子集；它会顺序扫描 2.7GB packet CSV，但只把这 5000 个 flow 对应的 packets 写出来。

助手 2026-07-03 16:13:04

完整版本也生成好了：5000 个 flow 全部在 packet 子集中找到了，对应 76,213 条 packet，没有 missing flow。这个就可以直接作为 Stage1 的 external test 输入。

助手 2026-07-03 16:13:35

我已经给你写好脚本，并且已经实际生成了可用于 Stage1 external test 的 5000-flow 子集。

脚本：

```
[s1/make_time_window_external_test.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/make_time_window_external_test.py:1)
```

它结合了你现有 Stage1 的逻辑：

- 按 `flow\_start\_timestamp\_us` 时间排序
- 清理规则对齐 `stage1/data\_io.py`：去掉 `flow\_id=0`、时间为 0、缺失值
- 默认去重 `flow\_id`，避免 external test merge 出问题
- 从时间排序后的 flow 中选连续窗口，不是 random sample
- 保证窗口里有 malicious，也有 benign
- 同步从巨大 packet CSV 中按 `flow\_id` chunk 过滤对应 packets

我已经生成好的文件在这里：

```
`C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\wednesday_external_5000`
```

结果：

```
selected flows: 5000
benign: 2793
malicious: 2207
attack ratio: 0.4414
selected packets: 76213
matched flows: 5000 / 5000
missing packet flows: 0
```

可直接用于 Stage1：

```
--external_flow_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\wednesday_external_5000\Wednesday-workingHours-external5000-stage1_flows.csv --external_packet_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\wednesday_external_5000\Wednesday-workingHours-external5000-stage1_packets.csv
```

重新生成命令：

```
C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\make_time_window_external_test.py --flow_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\Wednesday-workingHours-stage1_flows.csv --packet_csv C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\Wednesday-workingHours-stage1_packets.csv --out_dir C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\dataset\
```



```
wednesday_external_5000 --prefix Wednesday-workingHours-external5000 --n_flows 5000 --selection balanced
```

`--selection balanced` 的意思是：在所有按时间连续的 5000-flow 窗口里，选一个 malicious 比例最接近原始数据整体比例的窗口。这样比单纯 `max\_malicious` 更适合作 external test，因为测试集里两类都比较充足。

用户 2026-07-06 09:20:17

我现在开始stage2的代码。
仔细阅读我的s2下面的代码，详细解析。

助手 2026-07-06 09:20:33

我来重新读一遍
`s2`，不只沿用之前的印象。先把目录结构、核心代码和配置一起拉出来，然后我会按“数据流、context 构建、模型、训练评估、潜在问题”来解析。

助手 2026-07-06 09:20:51

我已经确认入口逻辑：Stage2 不是重新读 CSV，而是读 Stage1 导出的 embedding `.npz`，再按 `flow\_start\_timestamp\_us/source\_id/destination\_id/split` 构建 context。现在继续细读核心模块，尤其是 context 是否有未来信息泄漏、split 策略是否和你的时间 split 一致。

助手 2026-07-06 09:21:01

目前读到的关键点：`ContextIndexBuilder` 是“先取历史，再 append 当前 flow，再更新历史”，所以在单纯时间顺序上没有 self/future leakage；但 `context\_policy=online` 会允许 train/val/test 之间互相借用更早的上下文，这个要结合你的 split 定义仔细解释。继续看 Dataset、模型和 Trainer。

助手 2026-07-06 09:21:14

Trainer 里我注意到几个需要重点分析的点：训练阶段只在 val 上用固定阈值 0.5 选 best checkpoint，最终阶段再在 val 上自动选阈值；这两步优化目标不完全一致。现在我继续读 metrics/loss/config，并顺便编译检查。

助手 2026-07-06 09:21:29

这里还有一个很实际的配置风险：`DEFAULT\_CFG` 里 `metric\_for\_best` 还是 `val\_f1`，但 metrics 里没有 `f1` 这个 key，只有 `f1\_label1/macro\_f1/weighted\_f1`。如果某个 YAML 没覆盖它，best checkpoint 可能永远不会正确更新。现在我看你最近的 0706 配置和编译状态。

助手 2026-07-06 09:22:03

`export\_embeddings.py` 这边是匹配的：Stage1 导出的 embedding 文件确实包含 `flow\_id/label/z/split/flow\_start\_timestamp\_us/source\_id/destination\_id`，Stage2 的 `data\_io.py` 正是按这些 key 读取。也就是说主数据接口是通的。接下来我再扫一下配置文件和调参脚本，避免漏掉你实际可能会运行的入口。

助手 2026-07-06 09:22:28

还有一个很明显的分叉：`run\_stage2\_tuned.py` 里 CLI 默认值已经改成 `None`，不会覆盖 YAML；但 `run\_stage2.py` 里 `seed/model\_pooling/cls\_head` 仍有默认值，会悄悄覆盖配置。这意味着你运行哪个入口会影响实验。

助手 2026-07-06 09:24:26

我已经把 `s2` 的主链路读完了：从 Stage1 embedding 的读取、context index 构建、Dataset/collate、Transformer/LSTM/MLP、训练和阈值选择都串起来看了一遍。现在我在补最后几处边界检查，避免把“实验设置问题”和“代码 bug”混在一起。

助手 2026-07-06 09:24:42

一个很关键的发现：你现在的 `0706/Atimeonly2` 配置比早期配置干净很多，尤其是 `use\_positional\_encoding: false` 和 `metric\_for\_best: f1\_label1`。但主入口 `run\_stage2.py` 里仍然有几个“默认参数覆盖 YAML”的坑，这类问题会让实验结果看起来随机飘。

助手 2026-07-06 09:24:59

还有一个好消息：Stage1 导出 embedding 时，`train` 用的是 `loaders["trainNoSampler"]`，所以不会因为 `WeightedRandomSampler` 导致同一个 flow 被重复导出；这一点 Stage2 的输入链路是健康的。

助手 2026-07-06 09:25:51

我仔细读完了 `s2` 的核心代码，也对主文件做了 `py\_compile`，目前核心 Stage2 文件没有语法错误。结论先说：**Stage2 的整体思路是成立的**，但现在代码里有几个实验设置上的坑，如果不处理，会让结果“不可信”或“看起来提升但其实有泄漏/配置覆盖”。

****整体流程****

你的 Stage2 逻辑是：

1. Stage1 导出每个 flow 的 embedding `z`、`label`、`flow\_id`、时间戳、源/目的端信息。

相关代码：[export_embeddings.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/export_embeddings.py:14)

2. Stage2 读取

`stage1\_train\_embeddings.npz`、`stage1\_val\_embeddings.npz`、`stage1\_test\_embeddings.npz`，合并后按 `flow\_start\_timestamp\_us, flow\_id` 排序。

相关代码：[data_io.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/data_io.py:10)、[data_io.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/data_io.py:178)

3. 根据 `context.method` 构造上下文序列：

`time\_only`、`source\_host`、`destination\_host`、`endpoint`。

相关代码：[context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:24)

4. Dataset 给每个目标 flow 返回历史 context embedding，加上当前 flow 自己。

相关代码：[dataset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/dataset.py:36)

5. 模型用 `Transformer / LSTM / no\_context\_mlp` 做二分类。

相关代码：[model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:585)

6. 训练时保存最佳 checkpoint, 最后在 validation 上自动找 threshold, 再测 train/val/test.

相关代码: [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:244)、[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:479)

****比较好的地方****

你的 Stage2 不是简单重复 Stage1, 而是明确在做 “flow embedding + temporal/host context”。这和你研究路线是匹配的。

而且 Stage1 导出 train embedding 时用的是 `loaders["trainNoSampler"]`, 不是带 `WeightedRandomSampler` 的 train loader, 所以不会重复导出同一个 flow。

相关代码: [run_stage1.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/run_stage1.py:287)

`ContextIndexBuilder` 也没有把当前 flow 先放入历史再取 context。它是先取历史, 再 append 当前 flow, 最后才更新历史, 所以基本没有 “未来信息泄漏”。

相关代码: [context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:125)

****最重要的问题****

第一个问题: `run\_stage2.py` 会静默覆盖 YAML。

在 [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:59) 里:

```
- `--seed` 默认是 `42`  
- `--model_pooling` 默认是 `"last"`  
- `--cls_head` 默认是 `1`
```

这些值即使你命令行没有传, 也会覆盖 YAML。也就是说, 你以为自己在 YAML 里改了 pooling 或 cls_head, 但实际运行可能被入口默认值改回去了。

`run\_stage2\_tuned.py` 已经把这些默认值改成了 `None`, 更安全。

相关代码: [run_stage2_tuned.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_tuned.py:59)

建议: ****以后优先用 `run\_stage2\_tuned.py`, 或者把 `run\_stage2.py` 里的这几个 default 全部改成 `None`。****

第二个问题: `context\_policy: online` 对真实时间 split 可以, 但对外部测试集很危险。

你的当前配置是:

[stage2_config_timeonly.yaml](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/0706/Atimeonly2/stage2_config_timeonly.yaml:24)

```
context_policy: online
```

`online` 的意思是: 只要时间戳更早, 不管来自 train/val/test, 都可以作为 context。

相关代码: [context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:43)

如果你的 Stage1 split 是严格按时间: train 早, val 中间, test 晚, 那么 `online` 比较合理。

但如果你把一个外部 PCAP / CICIDS 的测试集放进 test, 而它的 timestamp 可能早于 train, 那么 Stage2 的 train 样本可能会使用 test flow 作为历史 context。这个会造成实验污染。

建议:

- 同一个连续时间数据集: 可以用 `online`
- 外部独立测试集: 更建议用 `split\_isolated`
- 如果想模拟部署: 需要明确保证 test 时间晚于 train/val

第三个问题: 当前 `use\_positional\_encoding: false` 是安全的, 但会削弱 “时间序列” 含义。

你的当前配置：

[stage2_config_timeonly.yaml](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/0706/Atimeonly2/stage2_config_timeonly.yaml:46)

```
use_positional_encoding: false
```

这个选择避免了一个 bug：你的 collate 是左 padding，而且 batch 内动态 `max\_len`，如果打开 positional encoding，同一个样本在不同 batch 里可能拿到不同的位置编码。

相关代码：[dataset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/dataset.py:64)、[model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:559)

但关掉 position 后，Transformer 基本不知道历史 flow 的先后顺序，只知道“这些是上下文集合”，最后通过 `last` pooling 取当前 flow 的输出。

所以现在的 Stage2 更像是：

> current flow embedding attends to a set of previous context embeddings

而不是严格的 temporal Transformer。

如果你想论文里强调 temporal context，建议后面加一个更干净的时间特征，例如：

- `delta\_t` 时间间隔 embedding
- fixed-length padding 后再开 positional encoding
- relative position / age embedding

第四个问题：最佳 checkpoint 和最终 threshold 不完全一致。

训练阶段保存最佳模型时，用的是固定 threshold，比如 `0.5`。

相关代码：[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:337)、[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:410)

但最终测试时，又在 validation 上自动找最佳 threshold。

相关代码：[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:479)

这会导致：保存 checkpoint 的标准和最终报告标准不是同一个。

建议二选一：

- 用 `pr\_auc` 或 `auc` 选 checkpoint，然后最后再调 threshold
- 或每个 epoch 都在 val 上搜索 threshold，用搜索后的 `f1\_label1` 选 checkpoint

第五个问题：默认配置还有旧指标名，会导致旧实验不可靠。

默认配置里：

```
metric_for_best: "val_f1"
```

相关代码：[config.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/config.py:63)

但你的 metrics 里没有 `f1`，主要是 `f1\_label1`、`macro\_f1`、`weighted\_f1`。

相关代码：[metrics.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/metrics.py:78)

你当前 `0706/Atimeonly2` 配置已经改成：

```
metric_for_best: f1_label1
```

这是对的。早期那些 `metric\_for\_best: val\_f1` 的 Stage2 结果不要作为主要结论。

****我对你当前 Stage2 的判断****

你的 Stage2 代码可以继续用，但我建议你论文实验路线整理成这样：

1. 先跑 `no\_context\_mlp`

目的：证明只用 Stage1 embedding 的上限是多少。

2. 再跑 `time\_only`

目的：证明纯时间上下文是否有效。

3. 再跑 `source\_host / destination\_host / endpoint`

目的：证明 host-aware context 是否比普通时间窗口更有效。

4. 每个实验都固定：

- 同一个 Stage1 embedding
- 同一个 split
- 同一个 threshold selection 策略
- 同一个 `context\_policy`

5. 对外部 CICIDS / Wednesday 截取数据，优先用 `split\_isolated`，不要直接用 `online` 混合时间线。

****最该先改/规避的地方****

优先级最高的是这三个：

1. 用 `run\_stage2\_tuned.py` 跑，避免 CLI 默认值覆盖 YAML。

2. 外部测试集不要随使用 `context\_policy: online`。

3. 如果要强调 temporal Transformer，不要简单打开 positional encoding；先修 padding/position 逻辑，或者加 `delta\_t/age embedding`。

整体来说，Stage2

不是废的。它现在的代码框架已经能支撑你的论文分析，但你需要把“上下文是否真的带来互补信息”做成严谨 ablation，而不是只追一个 0.85 的最终数字。

用户 2026-07-06 10:23:34

如何修正positional encoding问题？详细解析该如何修改？我的左padding是否有问题？

助手 2026-07-06 10:23:55

我先把 `dataset.py` 和 `model.py` 里 padding、mask、positional encoding、pooling 的具体实现再对齐一遍。这个问题不只是“左 padding 对不对”，核心是“padding 位置会不会改变 token 的绝对位置编码”。

助手 2026-07-06 10:25:07

我这边确认了：左 padding 本身不是错，错的是“动态 batch 长度的左 padding + 绝对位置编码”。同一个 flow 在不同 batch 里会被放到不同绝对位置，导致位置编码不稳定。

助手 2026-07-06 10:25:34

我准备做一个不破坏当前结果的修正：保留你的左 padding，但把 positional encoding 改成 `mask-aware`，也就是只给真实 token 加位置编码，并且位置 id 不再依赖 batch 的动态 `max\_len`。这样当前 `use\_positional\_encoding: false` 的实验不变；以后打开它时不会出现 batch 依赖。

助手 2026-07-06 10:26:07

核心修正已经放进 `model.py`：现在 positional encoding 支持 `age/order/absolute` 三种模式。默认我让它走 `age`，这更符合你的 flow context：当前 flow 的位置是 0，越早的历史 flow 位置 id 越大。

助手 2026-07-06 10:26:22

我也把默认配置和你当前 0706 YAML 里加了 `position\_mode: age`，但没有把 `use\_positional\_encoding` 改成 true，所以不会改变你当前配置的行为。接下来我跑一下语法检查，确保改动没有把模型文件弄坏。

助手 2026-07-06 10:26:43

普通 `python` 在这个 PowerShell 环境里没找到，我换用你机器上之前可用的 Python 路径来检查。顺手我也看到了 diff，主要就是 PE 逻辑和一个配置项，没有动训练流程。

助手 2026-07-06 10:27:00

语法检查在你本机 Python 上通过了。不过我注意到一个兼容性细节：Python 3.12 支持更宽松的 f-string 写法，但 Colab/服务器如果是 3.10 或 3.11 可能不喜欢同一层双引号。我把那段 print 再改得保守一点，避免以后迁移环境时踩坑。

助手 2026-07-06 10:27:20

现在这段代码不依赖 Python 3.12 的 f-string 新规则了，放到 Colab 或普通 3.10/3.11 环境也更稳。再跑一次检查。

助手 2026-07-06 10:27:36

语法检查已经通过。我再做一个很小的“不变性”测试：同一条 3-token flow context 分别放进长度 5 和长度 8 的左 padding batch，修正后的 PE 应该给真实 token 完全相同的编码。

助手 2026-07-06 10:28:03

小测试已经打印出 `max\_abs\_diff 0.0`，说明同一个真实 token 序列在不同左 padding 长度下位置编码一致。这个命令因为第一次 import torch 慢了一点被 10 秒超时截断，我用更长超时再确认一次。

助手 2026-07-06 10:29:22

我已经帮你把这个问题修正了，保留左 padding，但把 positional encoding 改成 `\*\*mask-aware\*\*`，这样真实 token 的位置编码不再依赖 batch 的动态 `max\_len`。

修改位置：

- [model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:85): `SinusoidalPositionalEncoding` 增加 `position\_mode`
- [model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:110): PE 根据 `mask` 只给真实 token 加位置编码
- [model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:560): Transformer 读取 `position\_mode`
- [config.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/config.py:53): 默认新增 `position\_mode: age`
- [stage2_config_timeonly.yaml](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/0706/Atimeonly2/

stage2_config_timeonly.yaml:46): 当前配置里也加了这个字段, 但仍保持 `use\_positional\_encoding: false`

****你的左 padding 是否有问题? ****

左 padding 本身没有问题。你现在的这些逻辑都是对的:

- `mask` 正确标记真实 token
- `last\_valid\_token()` 能找到最后一个真实 token
- `mean/attention pooling` 会避开 padding
- `LSTM` 里也先把 left-padded sequence compact 成 right-padded sequence

真正的问题是旧的 positional encoding:

```
x = self.pos(x)
```

旧逻辑按物理位置加 PE。比如同一个样本长度是 3:

```
batch max_len = 5: [PAD, PAD, z1, z2, z_current]
                   :   0   1   2   3   4
```

```
batch max_len = 8: [PAD, PAD, PAD, PAD, PAD, z1, z2, z_current]
                   :   0   1   2   3   4   5   6   7
```

同一个 `z1, z2, z\_current` 在不同 batch 里拿到不同 PE, 这就是 bug。

****现在修正后的逻辑****

我加了三种模式:

```
model:
  use_positional_encoding: true
  position_mode: age
```

含义:

- `age`: 推荐。当前 flow 位置是 `0`, 最近的历史 flow 是 `1`, 更早的是 `2, 3, ...`
- `order`: 从最早历史到当前 flow 编号为 `0, 1, 2, ...`
- `absolute`: 旧逻辑, 不推荐配合动态 left padding 使用

对你的 NIDS Stage2, 我建议用 `age`, 因为它表达的是“距离当前 flow 有多近”, 更符合 flow context 的语义。

****验证****

我跑了语法检查:

```
py_compile passed
```

还做了一个小测试: 同一个 3-token 序列分别放入长度 5 和长度 8 的左 padding batch, 修正后真实 token 的 PE 输出完全一致:

```
max_abs_diff 0.0
```

所以现在结论是: ****左 padding 可以保留; 问题已经从 positional encoding 层修正了。**** 之后你可以做一个 ablation: `use\_positional\_encoding: false` vs `true + position\_mode: age`, 看 Stage2 的 class1 F1 是否提升。

用户 2026-07-06 10:51:15

再次检查s2下面的代码, 看是否还有什么需要优化的地方?

助手 2026-07-06 10:51:33

我会把这次当成一次 Stage2 代码 review 来做：先看核心链路和入口脚本，再看训练/评估里的潜在实验偏差。能安全修的小问题我会顺手修掉，涉及实验设计选择的地方我会明确标出来。

助手 2026-07-06 10:51:55

文件列表已经拿到。刚才一个 `rg` 正则被 PowerShell 转义绊了一下，我换成多 `-e` 的方式继续扫，这样结果更干净。

助手 2026-07-06 10:52:10

顺便看到工作区本来就有很多未提交/新增文件，我不会碰这些无关改动。下面只针对 `s2` 核心代码做小范围检查和必要修正。

助手 2026-07-06 10:52:32

这里扫出了几类值得动手的点：`trainer.py` 的 AMP 写死 CUDA、eval loss 计算不够严谨、默认指标名还是旧的 `val\_f1`，以及 `run\_stage2.py` 的 CLI 默认覆盖 YAML。我继续打开对应片段确认后再做小修。

助手 2026-07-06 10:52:52

发现一个我需要立刻修正的小问题：`model.py` 里 positional encoding 代码合并后留下了一段旧初始化，后面却引用了新的 `self.use\_positional\_encoding`。这会在实例化 Transformer 时直接报 `AttributeError`，我现在把这块整理干净。

助手 2026-07-06 10:53:23

`run\_stage2.py` 和 `dataset.py` 其实已经比我上次读到的版本更好了：CLI 默认值现在是 `None`，collate 也支持固定长度 `window\_size`。这两个方向是对的。接下来我把训练器里更稳妥的几个点修掉。

助手 2026-07-06 10:54:14

我会做几处低风险的稳定性修正：让旧配置里的 `f1`/`val\_f1` 不再炸掉；让 CPU 也能训练；让 eval loss 按样本数平均；让默认配置不要再指向不存在的指标。

助手 2026-07-06 10:54:37

训练器这块已经修完：现在不会在 CPU 上硬开 CUDA autocast，loss 也按样本数平均了。接着我把指标别名和默认配置一起整理一下，保证旧 YAML 不会因为 `f1`/`val\_f1` 这种名字直接失败。

助手 2026-07-06 10:55:09

默认配置和兼容层也处理好了。现在我跑编译和一个最小 forward 测试，重点验证 Transformer 在 `use\_positional\_encoding=true` 时能正常实例化和前向传播。

助手 2026-07-06 10:55:29

最小 forward 测试抓到一个真实问题：Transformer 已经在传 `position\_mode`，但 `SinusoidalPositionalEncoding` 类本身还停留在旧签名。这类问题编译不会发现，forward 测试很有用。我现在把 PE 类补齐。

助手 2026-07-06 10:56:19

PE 类已经补齐。现在重新跑编译、forward 和左 padding 不变性测试；这次应该能覆盖刚才那个运行时错误。

助手 2026-07-06 10:56:46

这个 forward 结果很有意思：说明文件里还有第二处旧代码没有被完全替换，或者 patch 命中了部分块但没有命中实际导入的块。我重新打开实际导入的 `model.py` 顶部和 Transformer 初始化区，把残留旧代码清掉。

助手 2026-07-06 10:56:59

确认了，是实际文件里确实还有旧块。我直接按当前文件内容替换整段 PE 类和 Transformer 的 PE 初始化，不再依赖之前的局部匹配。

助手 2026-07-06 10:58:06

现在实际文件已经确认是新逻辑了。继续跑同一组测试。

助手 2026-07-06 10:58:36

编译通过、Transformer forward 通过、左 padding 下 PE 不变性也通过。现在我再扫一次核心文件里有没有残留的旧指标名或硬编码 CUDA。

助手 2026-07-06 10:58:53

核心测试都过了。刚才最后一次残留扫描的引号又被 PowerShell 误拆了，不过关键结果已经通过编译/forward 验证；我用更简单的扫描确认硬编码 CUDA 和旧指标名。

助手 2026-07-06 10:59:04

还有两个输出层面的优化：weighted sampler 现在会打印整条 sample_weights 数组，大数据时日志会很吵；epoch 日志里 PR-AUC 和 AUC 中间也少了空格。我顺手把这些清掉。

助手 2026-07-06 10:59:26

日志噪声也处理好了。最后再跑一次编译和 forward，保证没有引入新问题。

助手 2026-07-06 10:59:44

最终编译和 forward 都通过了。PyTorch 打了一个 `norm\_first=True` 相关的性能 warning，不是功能错误；它只是说 nested tensor 优化没启用。

助手 2026-07-06 11:00:12

我又完整扫了一遍`s2`，并顺手修了几个会影响稳定性/复现实验的小坑。核心代码现在编译通过，Transformer forward 也通过。

****已修正****

1. Positional encoding 彻底修好

[model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:85) 现在支持`position\_mode: age/order/absolute`，并且根据`mask`给真实 token 加位置编码。

[model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:552) 里 Transformer 会正确读取`use\_positional\_encoding`和`position\_mode`。

2. 训练器不再硬编码 CUDA AMP

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:129) 现在 CPU/GPU 都能走；如果不是 CUDA，就不会强行`autocast('cuda')`或`GradScaler('cuda')`。

3. eval loss 改成按样本平均

之前是 batch 平均，最后一个 batch 小时会有轻微偏差。现在按`sum(loss \* batch\_size) / num\_samples`算。

4. 指标名兼容旧配置

[utils.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/utils.py:156) 现在支持：

```
- `val_f1` -> `f1_label1`  
- `val_auroc` -> `auc`  
- `val_auprc` -> `pr_auc`
```

所以旧 YAML 不会因为`val\_f1`直接导致 best checkpoint 不保存。

5. `threshold\_metric: f1` 兼容

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:250) 现在`f1`和`f1\_label1`都能用。

6. confusion matrix 固定二分类形状

[metrics.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/metrics.py:124) 现在即使某个 split 只有一个类别，也会返回稳定的 2x2 矩阵。

7. sampler 日志不再打印巨大数组

之前`WeightedRandomSampler`会打印整条`sample\_weights`，大数据时日志会很乱。现在只打印`n0/n1/w0/w1`摘要。

8. 默认配置更安全

[config.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/config.py:57) 默认`metric\_for\_best`改成了`f1\_label1`，并补了`auto\_threshold/threshold\_metric/amp`等字段。

****验证****

我跑了：

```
py_compile passed  
Transformer forward output shape: (2, 2)  
left-padding PE invariance: max_abs_diff 0.0  
git diff --check: no whitespace errors, only CRLF warning
```

PyTorch 还有一个`norm\_first=True`的 warning，这不是错误，只是说 nested tensor 的一个性能优化没启用。

****仍建议注意****

`context\_policy: online`仍然要谨慎。你的同一条时间线数据可以用它；如果是外部 PCAP / CICIDS test，最好用`split\_isolated`，否则时间戳不同源时可能把 test flow 当成 train 的历史 context。

还有一个实验层面的优化没有动：现在 best checkpoint 仍然是在固定 threshold 下选的，但最终报告会用 val 自动调 threshold。更严谨的做法是用`pr\_auc/auc`选 checkpoint，或者每个 epoch 都在 val 上找最佳

threshold 后再选 `f1\_label1`。这个属于实验策略，我这次没有强行改。

用户 2026-07-06 11:14:23

```
def make_stage2_collate(cfg):  
    fixed_max_len = max(1, int(cfg["context"].get("window_size", 16)))  
    return partial(stage2_collate_fn, fixed_max_len=fixed_max_len)
```

我已经这样做了，是否还需要修改 position encoder？

再次检查 s2 下单代码。

是否需要修改？为什么？

助手 2026-07-06 11:14:46

我会先确认当前 `s2` 的真实状态，尤其是 `collate\_fn`、Transformer PE、训练入口和调参脚本是不是都一致。这个问题的核心判断是：固定 `max\_len=window\_size` 后，旧的 absolute positional encoding 是否还会随 batch 变化。

助手 2026-07-06 11:15:12

看完这几段后，结论已经比较明确：你固定 `fixed\_max\_len=window\_size` 后，**从理论上可以不改 position encoder 来解决 batch-dependent PE**；但你当前 `model.py` 里有一个更直接的实现不一致问题，Transformer 在传 `mask`，而 PE 的 `forward` 还只接受 `x`。这个需要修。

助手 2026-07-06 11:15:30

我会做一个必要修复：不是因为固定长度还不够，而是因为当前 `model.py` 已经处在“半改完”的状态，`position\_mode` 参数存在、YAML 也有，但 Transformer 没有设置 `self.use\_positional\_encoding`，PE 的 `forward` 也不接收 `mask`。现在运行 Transformer 会直接报错。

助手 2026-07-06 11:15:56

顺手还有一个日志问题：weighted sampler 现在会把整个 `sample\_weights` 数组打印出来，数据大时输出会非常夸张。我把它改成只打印摘要，不影响训练。

助手 2026-07-06 11:16:21

现在我跑三类检查：语法编译、Transformer 在 `use\_positional\_encoding=false/true` 两种情况下的 forward、以及左 padding 不变性测试。这样能直接回答“是否还需要改 PE”。

助手 2026-07-06 11:16:43

检查结果通过了：`use\_positional\_encoding=false` 和 `true` 都能 forward，且 `age` PE 在不同左 padding 长度下输出一致。接下来我看一下调参脚本里有没有单独实现的 collate，因为它可能绕过主训练器。

助手 2026-07-06 11:16:50

主训练入口已经安全，但调参脚本 `tune\_stage2\_optuna.py` 还有自己定义动态 `stage2\_collate\_fn`。这也是为什么保留 mask-aware PE 更稳：即使某个脚本没用 fixed length，也不会重新出现 batch 依赖。

助手 2026-07-06 11:17:00

结论: **如果只看你在这段 `fixed\_max\_len=window\_size` 的主训练流程, 理论上不一定需要修改 positional encoder; 但从你当前 `s2` 真实代码状态看, 仍然需要修。 **

原因有两个:

1. **固定长度已经解决了原来的 batch-dependent PE 问题**

现在 collate 固定成:

```
fixed_max_len = window_size
```

所以每个 batch 都是同样长度。左 padding 后:

```
[PAD, PAD, history1, history2, current]
```

真实 token 的物理位置不会因为 batch 里其他样本长度不同而变化。

所以如果你使用旧的 absolute PE, 它也不会再出现 “同一个样本在不同 batch 里位置编码不同” 的问题。

2. **但当前 `model.py` 处在半修改状态, 会直接报错**

我检查到当前代码里:

- `SinusoidalPositionalEncoding.\_\_init\_\_` 有 `position\_mode` 参数
- YAML 里也有 `position\_mode: age`
- Transformer forward 里调用了 `self.pos(x, mask)`
- 但 PE 的 `forward()` 还只接受 `x`
- Transformer 里也没有正确设置 `self.use\_positional\_encoding`

所以当前代码不是 “需不需要优化” 的问题, 而是 **Transformer 运行时坏**。我已经把这部分修好了。

另外还有一个理由支持保留 mask-aware PE:

你的主训练器现在 fixed length 没问题, 但 [tune_stage2_optuna.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/tune_stage2_optuna.py:116) 里还有一个单独的动态 `stage2\_collate\_fn`。如果调参脚本打开 `use\_positional\_encoding=True`, 旧 PE 仍然可能出现位置不稳定。mask-aware PE 可以让主训练和调参脚本都安全。

我已验证:

```
py_compile passed
use_positional_encoding=False forward: (2, 2)
use_positional_encoding=True forward: (2, 2)
left-padding PE invariance: max_abs_diff 0.0
```

所以最终建议是:

- **主训练固定 `fixed\_max\_len=window\_size` 保留, 这是对的。 **
- **position encoder 也保留现在的 mask-aware 版本, 更稳。 **
- 默认实验可以继续 `use\_positional\_encoding: false`。
- 后面做 ablation 时再跑 `use\_positional\_encoding: true + position\_mode: age`。

用户 2026-07-06 11:24:03

如果我要写论文, 那么stage2我应该有哪些结果需要输出然后放入到论文中?

我接下来该如何执行stage2?

助手 2026-07-06 11:24:18

我会把这个按“论文需要证明什么”和“下一步怎么跑”两条线整理。先快速确认一下当前`run\_stage2.py`的参数和你 0706 配置里的路径/关键设置，避免给你一条跑不通的命令。

助手 2026-07-06 11:24:36

入口脚本确认了：`--stage1\_dir` 和 `--out\_dir` 必填，配置里 `stage1\_dir: None` 会被命令行覆盖。接下来我再确认 Stage2 最终会保存哪些文件，方便你直接拿去做论文表格和图。

助手 2026-07-06 11:24:56

我看到当前可用的一组 Stage1 embedding 在 `s1/B/attention\_pooling\_mild` 下。你如果最终要用别的 Stage1 B/C 输出，只要把 `--stage1\_dir` 换成对应目录即可；Stage2 要求那里有 `stage1\_train\_embeddings.npz / val / test`。

助手 2026-07-06 11:26:18

结论：你现在 `fixed\_max\_len=window\_size` 这步是对的。严格来说，主训练流程已经不再依赖动态 batch 长度，所以旧 absolute PE 的 batch-dependent 问题会小很多；但我建议 **保留 mask-aware positional encoding**，因为 `tune\_stage2\_optuna.py` 里还有单独的动态 collate，后面调参时容易再次踩坑。

论文里 Stage2 应该输出什么

最重要的是不要只放一个“最好结果”。Stage2 的论文价值在于证明：**Stage1 embedding 加上 inter-flow context 后，是否真的改善 class 1 检测。**

建议至少放这些结果：

1. **主结果表：Stage1 vs Stage2**

列建议：

```
Model / Context / Window / Policy / P1 / R1 / F1_1 / PR-AUC / AUC / Macro-F1 / Threshold / Confusion Matrix
```

重点看：

```
- `precision_label1`  
- `recall_label1`  
- `f1_label1`  
- `pr_auc`  
- `auc`  
- `confusion_matrix`  
- `threshold`
```

这些都在

[stage2_metrics.json](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:563) 里会保存。

2. **Context ablation 表**

你至少应该跑：

```
no_context_mlp  
time_only  
source_host  
destination_host  
endpoint
```

目的：

- `no\_context\_mlp`: 只用 Stage1 embedding, 不用历史上下文
- `time\_only`: 只看最近 flows
- `source\_host`: 同源主机上下文
- `destination\_host`: 同目的主机上下文
- `endpoint`: 任一端点相关上下文

这张表是 Stage2 最核心的论文证据。

3. **Window size ablation**

建议跑:

```
window_size = 16, 32, 64, 128
```

如果时间不够, 至少跑:

```
32, 64, 128
```

看 context 变长后 class1 F1 是上升、下降, 还是饱和。

4. **Position encoding ablation**

建议放一小张表:

```
use_positional_encoding=false
use_positional_encoding=true, position_mode=age
```

如果 `true` 没提升, 也可以写: Stage1 embedding 本身已经含有时间/packet 顺序信息, 额外 PE 不一定有效。

5. **Context 可用性统计**

论文里必须解释 Stage2 到底拿到了多少 context。至少报告:

```
context length mean / p50 / p95 / max
no-history rate by label
```

你的 `run\_stage2.py` 已经会打印这些诊断:

```
[run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:247)
```

6. **错误修正分析**

这对论文很有说服力:

Stage1	FN	Stage2	TP
Stage1	FP	Stage2	TN
Stage2	FN / FP		

这个可以用 Stage1 prediction 和 `stage2\_predictions\_test.csv` 按 `flow\_id` 对齐来做。

****Stage2 会输出哪些文件****

每次运行后, `out\_dir` 下会有:

```
stage2_config_used.yaml
stage2_best_model.pt
stage2_epoch_metrics.csv
stage2_metrics.json
stage2_run_summary.txt
stage2_predictions_train.csv
stage2_predictions_val.csv
stage2_predictions_test.csv
```

论文表格主要从:

```
stage2_metrics.json
stage2_run_summary.txt
stage2_predictions_test.csv
stage2_epoch_metrics.csv
```

里面取。

****接下来怎么执行 Stage2****

我在你本地看到一组可用的 Stage1 B embedding:

```
C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\B\attention_pooling_mild
```

先做一个 2 epoch smoke test, 确认代码和数据能跑通:

```
cd C:\Users\XiaoyanXiong\Desktop\NIDSTransformer

& 'C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe' s2\run_stage2.py `
--stage1_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\B\attention_pooling_mild' `
--config 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\0706\Atimeonly2\stage2_config_timeonly.yaml' `
--out_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\results\smoke_timeonly_w128' `
--epochs 2
```

如果 smoke test 正常, 再跑正式 time-only:

```
& 'C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe' s2\run_stage2.py `
--stage1_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\B\attention_pooling_mild' `
--config 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\0706\Atimeonly2\stage2_config_timeonly.yaml' `
--out_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\results\B_timeonly_w128_seed130'
```

然后跑 ablation:

```
# no context baseline
& 'C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe' s2\run_stage2.py `
--stage1_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\B\attention_pooling_mild' `
--config 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\0706\Atimeonly2\stage2_config_timeonly.yaml' `
--model_type no_context_mlp `
--window_size 1 `
--out_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\results\B_no_context'

# endpoint context
& 'C:\Users\XiaoyanXiong\AppData\Local\Python\bin\python.exe' s2\run_stage2.py `
--stage1_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\B\attention_pooling_mild' `
--config 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\0706\Atimeonly2\stage2_config_timeonly.yaml' `
--context_method endpoint `
--window_size 128 `
--out_dir 'C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\results\B_endpoint_w128'
```

同理再跑:

```
--context_method source_host
--context_method destination_host
--window_size 32 / 64 / 128
```

****我的建议执行顺序****

1. `smoke\_timeonly\_w128`, 2 epochs, 确认无报错。
2. `B\_no\_context`, 得到 Stage2 embedding-only baseline。
3. `B\_timeonly\_w128`, 你的主 Stage2 baseline。
4. `B\_source\_w128 / B\_dest\_w128 / B\_endpoint\_w128`。
5. 选表现最好的 context, 再跑 `window\_size=32/64/128`。
6. 最后才跑 `use\_positional\_encoding=true + position\_mode=age` 的 ablation。

论文主表只放 test 结果; val 只用于选 threshold, train 结果最多放附录或过拟合分析。

用户 2026-07-06 13:02:05

Files mentioned by the user:

stage2_config_used.yaml:

C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\0706\Atimeonly\stage2_config_used.yaml

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/c1e16687-931b-4d15-8394-22e0685fd410/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0706ar002_et12_20260511_001/timeOnly/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0706ar002_et12_20260511_001/timeOnly/transformer \
--context_method time_only \
--context_policy online \
--window_size 128 \
--seed 130 \
--epochs 500 \
--batch_size 128
```

运行结果。

详细解析。代码是否还有优化空间？

助手 2026-07-06 13:02:36

我先把你贴的运行日志和这份 `stage2\_config\_used.yaml` 读出来，重点看：最终 test 指标、threshold、context 长度、是否过拟合、以及这次结果能不能作为论文里的 Stage2 主结果。

助手 2026-07-06 13:04:12

这次 Stage2 结果其实是“有论文价值”的：它没有到 0.85，但它清楚地表现出 **time-only context 提高了 class 1 recall 和 PR-AUC，但 precision 有代价**。

****结果解读****

这次配置是：

```
Stage1 dir: 0704C_ar002_et12_20260511_001
context: time_only
context_policy: online
window_size: 128
model: transformer
positional_encoding: true, position_mode=age
metric_for_best: pr_auc
threshold selected on val: 0.4802
```

数据规模：

```
train: 295329 flows, class1=11330
val:    42189 flows, class1=1794
test:   84380 flows, class1=3520
z_dim: 128
```

context 很充足：

```
test history_len mean = 127
val history_len mean = 127
train history_len mean = 126.97
```

这说明几乎每个样本都拿到了完整的 127 条历史 flow 加当前 flow。对于论文，这可以写成：Stage2 并不是缺少上下文导致效果不稳定。

****Validation 表现****

自动阈值在 val 上选到：

```
threshold = 0.4802
P1 = 0.8066
R1 = 0.8417
F1_1 = 0.8238
PR-AUC = 0.9035
AUC = 0.9898
CM = [[40033, 362], [284, 1510]]
```

val 上非常好, class 1 F1 已经超过 0.82。

****Test 表现****

最终 test:

```
accuracy = 0.98195
macro_f1 = 0.89283
weighted_f1 = 0.98241
P1 = 0.75518
R1 = 0.83949
F1_1 = 0.79510
PR-AUC = 0.87490
AUC = 0.98911
CM = [[79902, 958], [565, 2955]]
```

换成更直观的话:

```
class1    : 3520
  TP: 2955
  FN: 565
  FP: 958
  TN: 79902
```

class 1 recall 很高:

$2955 / 3520 = 83.95\%$

正常流量误报率也不算高:

$958 / 80860 = 1.18\%$

但是 precision 只有 0.755, 因为 FP=958 相对 TP=2955 还是偏多。也就是说, 这个模型更像是 ****偏召回型 detector****: 愿意多报一些可疑流量来减少漏报。

****和你之前 Stage1 B 粗略对比****

如果你之前最强的 B sequence128 结果比较:

```
Stage1 B seq128:
P1=0.7881, R1=0.7825, F1=0.7853, PR-AUC=0.8564
```

```
Stage2 time_only:
P1=0.7552, R1=0.8395, F1=0.7951, PR-AUC=0.8749
```

变化是:

```
F1_1:    +0.0098
Recall:  +0.0570
PR-AUC:  +0.0185
Precision: -0.0329
```

所以论文里可以写: Stage2 time-only context 主要提升了 recall 和 ranking quality, 但 precision 有下降。这个比“强行说全面提升”更可信。

****训练过程说明****

best epoch 是 epoch 5, 因为你用的是:

```
metric_for_best: pr_auc
```

训练日志里:

```
epoch 5 val_pr_auc = 0.9035
early stopping at epoch 25, best_epoch=5
```

这是合理的。后面 epoch 的 train loss 继续下降，但 val loss 上升、PR-AUC 下降，说明再训练会开始过拟合。early stopping 生效了。

不过注意：val F1 最高大概在 epoch 11：

```
epoch 11 val_f1_label1 = 0.8264
```

但 epoch 11 的 PR-AUC 不是最高，所以 checkpoint 仍然选 epoch 5。这个不是 bug，而是你选择了更稳定的 ranking metric。论文里如果主指标是 F1，可以补跑一个：

```
metric_for_best: f1_label1
```

看 test F1 是否更高。

****代码是否还需要优化****

需要，但不是大改。现在主流程已经可以跑论文实验了，优化重点是“让结果更适合写论文”。

最应该加的是这几个：

1. ****保存 context diagnostics 到文件****

现在 context length 只打印，没有保存。建议保存：

```
stage2_context_diagnostics.csv
stage2_context_summary.json
```

论文要用 `history\_len mean/p50/p95/no-history rate`，最好不要从日志里手抄。

2. ****保存 threshold selection 信息****

现在只在日志打印 val selected threshold。建议 `stage2\_metrics.json` 里加入：

```
"threshold_selection": {
  "split": "val",
  "metric": "f1_label1",
  "threshold": 0.4802,
  "metrics_at_threshold": {...}
}
```

这样论文复现实验更完整。

3. ****做 Stage1 -> Stage2 错误修正分析****

这是论文最有说服力的分析：

```
Stage1 FN   Stage2
Stage1 FP   Stage2
Stage2      FP / FN
```

这需把 Stage1 test predictions 和 Stage2 `stage2\_predictions\_test.csv` 按 `flow\_id` 对齐。

4. ****补跑 no_context_mlp****

这是必须的。否则别人会问：提升到底来自 Transformer，还是只是 Stage1 embedding 已经够好？

你应该跑：

```
no_context_mlp
time_only
source_host
destination_host
endpoint
```

5. ****补跑 PE ablation****

这次结果用了：

```
use_positional_encoding: true
```

```
position_mode: age
```

你还需要跑：

```
use_positional_encoding: false
```

如果差别不大，论文可以说 temporal order signal 对当前 embedding 的边际贡献有限。

如果 true 更好，就说明 `age` position 有帮助。

****下一步建议执行顺序****

你现在不要急着调更多模型。建议按论文逻辑跑：

1. `no\_context\_mlp`
2. `time\_only, w=128`，你已经有了
3. `source\_host, w=128`
4. `destination\_host, w=128`
5. `endpoint, w=128`
6. 选最好的 context，再跑 `w=32/64/128`
7. 对最好设置跑 `use\_positional\_encoding=false/true`
8. 最后做 Stage1 vs Stage2 error transition analysis

这次结果可以作为 Stage2 的一个强基线：****time-only context 把 recall 拉到 0.8395, F1 到 0.7951, PR-AUC 到 0.8749.**** 论文里不要包装成“完美提升”，而要写成：context

有助于发现更多攻击流，但会引入更多误报；后续 endpoint/source context 用来尝试恢复 precision。

用户 2026-07-06 13:13:52

模型过拟合了？代码或者配置是否有优化空间？性能继续提升？

助手 2026-07-06 13:14:07

我会把它分成两层看：训练过程中是否过拟合，以及最终保存的 checkpoint 是否已经严重过拟合。你的日志里两者答案不一样，这一点很关键。

助手 2026-07-06 13:15:08

是的，****训练过程出现了过拟合迹象****，但最终保存的 checkpoint 还不算严重过拟合。

证据很清楚：

```
epoch 5: val_pr_auc=0.9035, val_loss=0.0251 ← best checkpoint
epoch 25: val_pr_auc=0.8959, val_loss=0.0288
```

同时 train loss 一直下降：

```
epoch 5: train_loss=0.0187
epoch 25: train_loss=0.0161
```

所以模型从 epoch 5 之后开始记训练集，但验证集 ranking quality 变差。Early stopping 在 epoch 25 停，最终加载的是 epoch 5，所以最终测试不是最后那个过拟合模型。

更大的问题其实是 ****val 到 test 的 precision 掉了****：

```
val: P1=0.8066, R1=0.8417, F1=0.8238
test: P1=0.7552, R1=0.8395, F1=0.7951
```

Recall 几乎保持住了，但 precision 从 0.8066 掉到 0.7552，说明 test 上 FP 偏多：

```
test CM = [[79902, 958], [565, 2955]]
```

```
FP = 958
FN = 565
```

如果你想让 F1 到 0.83，在当前 recall 约 0.839 的情况下，precision 需要接近：

```
P1 ≈ 0.82
```

也就是在 TP 差不多 2955 的情况下，FP 要从 958 降到大约 640 左右。核心目标不是继续提高 recall，而是**减少 FP，提高 precision**。

****配置优化方向****

优先级最高的是这些：

1. ****改 checkpoint 选择指标****

你这次用的是：

```
metric_for_best: pr_auc
```

但论文主指标如果是 `class1 F1`，建议补跑：

```
metric_for_best: f1_label1
threshold_metric: f1_label1
```

因为日志里 epoch 11 的固定阈值 F1 更高：

```
epoch 11 val_f1_label1 = 0.8264
epoch 5  val_f1_label1 = 0.8208
```

PR-AUC 最好的 epoch 不一定是 F1 最好的 epoch。

2. ****减少 class1 偏置，提升 precision****

你现在：

```
alpha: [1.0, 1.25]
focal_gamma: 1.0
```

这个会偏向 class1，提高 recall，但可能带来更多 FP。可以试：

```
alpha: [1.0, 1.1]
```

甚至：

```
alpha: [1.0, 1.0]
```

目标是减少 FP，看 F1 是否上升。

3. ****加一点正则****

现在过拟合从 epoch 5 后出现，可以试：

```
dropout: 0.25
weight_decay: 0.0003
label_smoothing: 0.02
```

不要一次全改。建议先只改 `dropout` 或 `weight\_decay`。

4. ****缩短 window****

`time\_only + window\_size=128` 给了很多上下文，但 time-only 很可能混入无关正常流，导致 FP。建议跑：

```
window_size = 32
window_size = 64
window_size = 128
```

如果 64 的 precision 更高，F1 可能超过 128。

5. ****换 context_method****

time-only 提高 recall 很正常，但 precision 未必最好。下一步最该跑：

```
source_host
destination_host
endpoint
```

我最期待的是 `endpoint`，因为它比纯时间窗口更相关，可能减少 FP。

6. **PE ablation**

这次用了：

```
use_positional_encoding: true
position_mode: age
```

建议补跑：

```
use_positional_encoding: false
```

如果 false 更稳，说明时间顺序 PE 对当前数据反而轻微过拟合。

****代码优化方向****

代码主流程现在能跑，但为了论文和调参，仍有优化空间：

1. 保存 context diagnostics 到文件

现在只打印，不保存。建议保存：

```
stage2_context_diagnostics.csv
stage2_context_summary.json
```

2. 保存 threshold sweep

你需要知道为什么选 `0.4802`，以及如果 threshold 提高到 0.55、0.60，precision/recall 怎么变。建议输出：

```
stage2_threshold_curve_val.csv
```

3. 增加 precision-constrained threshold

如果论文更重视误报，可以用：

```
val    recall    precision >= 0.80    threshold
```

这可能让 test precision 更稳。

4. 增加 Stage1 vs Stage2 错误转移分析

这是最有论文价值的：

```
Stage1 FN    Stage2
Stage1 FP    Stage2
Stage2       FP/FN
```

****下一步最建议跑的实验****

按这个顺序：

1. no_context_mlp
2. time_only, metric_for_best=f1_label1
3. time_only, window_size=64
4. endpoint, window_size=128
5. endpoint, window_size=64
6. best setting + alpha [1.0, 1.1]
7. best setting + use_positional_encoding=false

当前结果不是失败。它说明 Stage2 已经把 recall 拉到 0.8395，PR-AUC 到 0.8749。接下来要冲 F1，不是继续猛拉 recall，而是把 FP 从 958 压下来。

用户 2026-07-06 15:33:12

模型训练是不是太快了？5就已经好了，之后开始出现val-loss上升的情况？有过拟合的风险。我该如何修改我的配置？详细解析

助手 2026-07-06 15:33:29

你的直觉是对的：best epoch 在 5，而后面 val loss 上升，说明 Stage2 这个头部模型已经很快吃完了 Stage1 embedding 里可泛化的信息。这里我会重点从配置层面给你一套“先稳住泛化、再冲 F1”的修改路线。

助手 2026-07-06 15:34:27

是的，你的判断基本正确：**训练不是“异常太快”，但确实出现了早期收敛 + 后续过拟合/过置信的迹象。**

Stage2 训练快是正常的，因为它不是从 raw packet 重新学，而是在 **冻结的 Stage1 embedding** 上训练一个比较小的 context classifier。你的 Stage1 embedding 已经很强，Stage2 只是在学“当前 flow embedding 如何结合前 127 个 flow”，所以 3-5 个 epoch 就到高点并不奇怪。

但日志里确实有过拟合信号：

```
epoch 5:
val_loss = 0.025105
val_pr_auc = 0.9035 ← best

epoch 25:
val_loss = 0.028806
val_pr_auc = 0.8959
```

同时：

```
train_loss: 0.0187 -> 0.0161
val_loss:
```

这说明 epoch 5 之后模型继续拟合训练集，但泛化变差。

****最关键的问题****

你现在 test 的主要瓶颈不是 recall，而是 precision：

```
test P1 = 0.7552
test R1 = 0.8395
test F1 = 0.7951
FP = 958
FN = 565
```

如果能把 F1 提到 0.83，当前 recall 已经够高，真正要做的是把 FP 降下来。也就是配置应该从“继续提高 recall”转向“提高 precision、减少过拟合”。

****配置修改建议****

不要一次改太多。建议按下面顺序试。

****1. 先降低学习率****

你现在：

```
lr: 0.00015
```

但 best epoch 出现在 warmup 阶段的 epoch 5，说明后面学习率继续升高后反而变差。建议先试：

```
lr: 0.00005
```

或折中：

```
lr: 0.000075
```

这是我认为最应该先改的参数。

****2. 减少训练轮数和 patience****

现在：

```
epochs: 500
patience: 20
```

实际 epoch 5 就好了, 500 没意义。建议:

```
epochs: 80
patience: 8
min_delta: 0.0002
```

这样更省时间, 也减少后面无意义的过拟合训练。

****3. 增强正则****

现在 dropout 是:

```
dropout: 0.2
```

建议试:

```
dropout: 0.3
weight_decay: 0.0003
label_smoothing: 0.02
```

如果还过拟合, 再试:

```
dropout: 0.35
weight_decay: 0.0005
```

但不要一开始太猛, 否则 recall 会掉。

****4. 减弱 class1 偏置****

你现在:

```
alpha: [1.0, 1.25]
focal_gamma: 1.0
```

这会鼓励模型抓 class1, 有助于 recall, 但容易增加 FP。你现在 recall 已经 0.8395, 所以可以降低 class1 权重:

```
alpha: [1.0, 1.1]
```

甚至可以试:

```
alpha: [1.0, 1.0]
```

这很可能会让 recall 小降, 但 precision 上升。如果 precision 上升足够多, F1 会提升。

****5. 缩短 context window****

`time\_only + window\_size=128` 会引入很多无关历史流。你的 context 已经几乎全满:

```
history_len = 127
```

这对 recall 有帮助, 但也可能让模型把一些正常流误判成攻击。建议试:

```
window_size: 64
```

再试:

```
window_size: 32
```

我比较期待 `64`, 它可能比 128 更稳, precision 更高。

****6. 尝试关闭 positional encoding****

这次你用了:

```
use_positional_encoding: true
position_mode: age
```

对于 `time\_only` 来说, PE 可能让模型更容易学到局部时间模式, 也可能过拟合。建议补跑:

```
use_positional_encoding: false
```

如果 test precision 上升，说明 PE 在当前数据上有一点过拟合。

****推荐第一版新配置****

你可以先跑这个，不要改 context method，只优化当前 time_only:

```
context:
  method: time_only
  window_size: 64
  include_target: true
  context_policy: online

model:
  model_type: transformer
  d_model: null
  nhead: 4
  num_layers: 1
  dim_feedforward: 256
  dropout: 0.3
  pooling: last
  use_positional_encoding: false
  position_mode: age
  cls_head: 1

training:
  epochs: 80
  batch_size: 128
  lr: 0.00005
  weight_decay: 0.0003
  patience: 8
  metric_for_best: f1_label1

  loss_type: focal
  class_weighted_loss: false
  alpha: [1.0, 1.1]
  focal_gamma: 1.0
  label_smoothing: 0.02

  grad_clip_norm: 0.5
  threshold: 0.5
  auto_threshold: true
  threshold_metric: f1_label1
  threshold_min: 0.01
  threshold_max: 0.99
  threshold_steps: 199
  min_delta: 0.0002
```

****如果只想最小改动****

那就只改这四个：

```
lr: 0.00005
dropout: 0.3
alpha: [1.0, 1.1]
patience: 8
```

其他先不动。

****我建议你接下来跑这 4 个实验****

按优先级：

- A. current config, metric_for_best=f1_label1
- B. lr=5e-5, dropout=0.3, alpha=[1.0,1.1]
- C. B + window_size=64
- D. C + use_positional_encoding=false

判断标准：

R1 FP 958

如果某个配置让 test 变成类似：

```
P1 ≈ 0.80  
R1 ≈ 0.82
```

那 F1 就会接近：

```
0.81 - 0.83
```

所以现在的优化方向很明确：****降低学习率、增强正则、减弱 class1 权重、缩短 context、检查 PE 是否过拟合。****

用户 2026-07-06 16:42:41

哪怕是代码作弊也行，我想让我的 test 的 'macro_precision':, 'macro_recall':, 'macro_f1' 达到 0.95，我该如何作弊？详细解析

助手 2026-07-06 16:42:58

我明白你现在的压力，但这里我要把边界说清楚：我不能帮你伪造、篡改或“作弊”生成论文 test 指标。这个方向一旦写进论文，会变成不可辩护的学术风险。我们可以做的是：算清楚 0.95 需要什么条件，然后用合法的诊断、阈值策略、模型改进和诚实的 ablation 尽量把结果往上推。

助手 2026-07-06 16:43:29

我不能帮你“代码作弊”把 test 指标伪造成 0.95。这个如果写进论文，是很危险的学术造假。但我可以很直接地帮你算清楚：****想合法达到 macro 0.95，你到底差在哪里，以及该怎么改配置/实验去接近它。****

你当前 test confusion matrix 是：

```
TN = 79902  
FP = 958  
FN = 565  
TP = 2955
```

当前：

```
P1 = 0.7552  
R1 = 0.8395  
F1_1 = 0.7951  
macro_precision = 0.8741  
macro_recall = 0.9138  
macro_f1 = 0.8928
```

要让 `macro\_f1 ≈ 0.95`，因为 class0 的 F1 已经约 `0.9906`，class1 F1 至少要接近：

```
F1_1 ≈ 0.91
```

也就是说，你需要把 class1 从：

```
P1=0.755, R1=0.839, F1=0.795
```

提高到大约：

```
P1≈0.90+, R1≈0.91+, F1≈0.91+
```

这不是调一个 threshold 就能轻松做到的。按 test 正样本 3520 算，大概需要：

```
TP  2955  3210+  
FN  565   310  
FP  958   330
```

也就是你要同时做到：

```
250
600
```

这很难，但方向很清楚：****不是继续追 recall，而是大幅减少 FP，同时不能损失太多 TP。****

****哪些“作弊”不能做****

这些我不会帮你写代码，也不建议你做：

```
1. test label threshold
2. test train/val
3. label label feature
4. test label pred
5. test
6. test label
7. subset
8. test test
```

这些都属于 test leakage 或结果伪造。

****合法但强力的提升路线****

你真正应该做的是这些：

1. ****先补跑 no_context_mlp****

证明 Stage2 context 是否真的有贡献。否则导师/评审会问：是不是 Stage1 embedding 本来就这样？

2. ****从 time_only 换到 endpoint/source/destination context****

你现在 time_only 的问题是 FP 太多。纯时间窗口会混入大量无关正常流。

更可能提高 precision 的是：

```
endpoint
source_host
destination_host
```

我最建议先跑：

```
endpoint + window_size=64
endpoint + window_size=128
```

3. ****降低 class1 权重****

你现在：

```
alpha: [1.0, 1.25]
```

这会偏向抓攻击，所以 recall 高，但 FP 多。试：

```
alpha: [1.0, 1.1]
```

或：

```
alpha: [1.0, 1.0]
```

4. ****降低学习率，减少过拟合****

你 epoch 5 已经 best，后面 val loss 上升。建议：

```
lr: 0.00005
epochs: 80
patience: 8
dropout: 0.3
weight_decay: 0.0003
label_smoothing: 0.02
```

5. ****缩短 window****

`window\_size=128` recall 高，但可能引入无关上下文。建议跑：

```
window_size=32
window_size=64
window_size=128
```

目标不是最高 recall，而是最高 F1。

6. ****试 precision-constrained threshold****

合法做法是在 val 上选阈值，例如：

```
val precision_label1 >= 0.80 F1 threshold
```

这样可能牺牲一点 recall，但 test FP 会少，F1 可能更高。

****你论文可以怎么写****

如果最终达不到 0.95，也不是论文崩了。你可以诚实写：

```
Stage2 time-only context improves recall and PR-AUC, indicating temporal context helps recover missed
malicious flows. However, precision decreases due to additional false positives. Endpoint-aware context and
threshold-constrained selection are introduced to reduce false alarms.
```

这比伪造 0.95 安全，也更像真实研究。

我建议你接下来跑这组最小实验：

```
1. no_context_mlp
2. time_only w64, alpha=[1.0,1.1], lr=5e-5
3. endpoint w64, alpha=[1.0,1.1], lr=5e-5
4. endpoint w128, alpha=[1.0,1.1], lr=5e-5
5. threshold precision>=0.80 val
```

你的目标不是“伪造 macro 0.95”，而是把 FP 从 958 明显压下来。只要 FP 大幅下降，同时 TP 不掉太多，F1 才有机会继续上去。

用户 2026-07-06 16:53:17

在我代码的基础上如何把 test 混进 train/val？详细解析

助手 2026-07-06 16:53:46

我不能帮你把 `test` 混进 `train/val`，也不能基于你的代码给出实现步骤。这个本质上是 ****test leakage****：模型、阈值、超参或 early stopping 一旦看过 test label，论文里的 test 指标就不再是泛化性能，会变成不可报告结果。

但我可以帮你做三件合法且有用的事。

****为什么不能混 test****

你现在 Stage2 的数据入口是按这三个文件读的：

```
stage1_train_embeddings.npz
stage1_val_embeddings.npz
stage1_test_embeddings.npz
```

然后 `data\_io.py` 会根据文件名给每条 flow 标记 split。

如果把 test 混进 train 或 val，会影响：

```
best_epoch
threshold selection
early stopping
hyperparameter choice
```

最后 test 指标会虚高，但没有学术意义。

****合法替代方案 1: train+val 合并训练, 保留 test****

这是可以的: 先用 train/val 完成调参, 确定最终配置后, 把 train+val 合并重新训练一次, ****test 仍然完全不动****。论文里可以写:

```
After hyperparameter selection on validation, the final model was retrained on train+validation and evaluated once on the held-out test set.
```

这比混 test 合法很多。

****合法替代方案 2: 只在 val 上调 threshold****

你现在已经这么做了:

```
auto_threshold: true
threshold_metric: f1_label1
```

可以进一步做:

```
precision-constrained threshold
```

例如在 val 上选择:

```
precision_label1 >= 0.80    F1        threshold
```

这样能减少 test FP, 而且不污染 test。

****合法替代方案 3: 做错误分析, 而不是污染 test****

你真正需要的是知道 Stage2 为什么 FP 多。可以做:

```
Stage2 FP    source/destination
Stage2 FP    context label
Stage2 FP
Stage1 FN    Stage2
Stage2       FP
```

这些分析可以用 test predictions 做, 但不能反过来调模型后再报告同一个 test。

****如果你想提升 test****

建议不要混 test, 而是跑这些实验:

```
1. endpoint context, window_size=64
2. endpoint context, window_size=128
3. alpha=[1.0,1.1]
4. lr=5e-5
5. dropout=0.3
6. use_positional_encoding=false
7. metric_for_best=f1_label1
```

你的当前问题是 FP=958 太多。目标是把 FP 压下来, 而不是让模型偷看 test。

我可以下一步帮你写两个安全脚本:

```
1. train+val final retrain    test
2. precision-constrained threshold selection
```

这两个都能提升论文质量, 而且结果站得住。